Rockchip Linux PCIe 开发指南

文件标识：RK-KF-YF-141

发布版本：V7.0.0

日期：2026-03-05

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

产品版本

芯片名称	内核版本
RK1808	4.4， 4.19
RK3528	4.19，5.10，6.1，6.12
RK3562	5.10，6.1，6.12
RK3566/RK3568	4.19，5.10，6.1，6.12
RK3572	6.12
RK3576	6.1，6.12
RK3588	5.4，5.10，6.1，6.12

RK3399使用不同的PCIe控制器IP，不在本文档覆盖范围，请参考《Rockchip_RK3399_Developer_Guide_PCIe_CN》。

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	林涛	2021-01-15	初始版本
V1.1.0	林涛	2021-01-22	增加PCIe 3.0控制器异常情况的检查信息
V1.2.0	林涛	2021-01-26	增加PCIe 2.0 Combo phy异常排除信息
V1.3.0	林涛	2021-02-04	增加MSI和MSI-X支持数量的问题描述
V1.4.0	林涛	2021-02-05	增加地址分配异常信息
V1.5.0	林涛	2021-02-06	增加PCIe2x1的PHY支持SSC说明
V1.6.0	林涛	2021-02-23	增加MSI/MSI-X调试支持和运行态设备异常说明
V1.7.0	林涛	2021-02-26	增加Legacy INT的说明
V1.8.0	林涛	2021-02-27	增加标注EP功能件开发说明
V1.9.0	林涛	2021-03-16	增加FW存在异常设备的说明
V2.0.0	林涛	2021-04-12	增加用户态访问异常说明
V2.1.0	林涛	2021-04-21	增加PCIe转XHCI芯片异常说明
V2.2.0	林涛	2021-04-23	增加lane拆分复位IO说明以及休眠唤醒异常说明
V2.3.0	林涛	2021-05-12	增加lane拆分下电源配置说明
V2.4.0	林涛	2021-06-04	增加LTSSM附录以及新增debug项说明
V2.5.0	林涛	2021-06-08	增加PCIe 3.0驱动因PHY异常而退出的说明
V2.6.0	林涛	2021-07-07	增加PCIe 地址空间配置说明

版本号	作者	修改日期	修改说明
V2.7.0	林涛	2021-07-14	增加PCIe设备重扫描说明
V2.7.1	林涛	2021-07-15	常见应用问题章节的格式修订
V2.8.0	林涛	2021-07-23	修订开发资源获取地址
V2.9.0	林涛	2021-07-29	增加legacy中断异常以及IO端口访问需求说明
V3.0.0	林涛	2021-08-02	增加PCIe 2.0 combo phy的充电检测异常处理方法
V3.1.0	林涛	2021-08-23	增加PCIe 3.0电源配置对时钟芯片的影响说明
V3.2.0	林涛	2021-09-01	增修PCIe BAR地址空间在64位和32位模式的说明
V3.3.0	林涛	2021-09-09	增加FW报错log以及可能的修复手段
V3.4.0	林鼎强	2021-09-18	增加设备IO BAR地址空间配置异常的说明
V3.5.0	林涛	2021-09-23	增加PCIe内存访问性能的说明
V3.6.0	林涛	2021-11-03	增加DMA支持说明
V3.7.0	林涛	2021-11-25	增加标准EP的bar空间异常说明
V4.0.0	杨凯	2021-12-20	增加RK3588说明
V4.1.0	林涛	2022-01-28	增加present信号， 线程初始化说明以及去除combophy补丁说明
V4.2.0	林鼎强	2022-03-21	更新“芯片互联功能”描述
V4.3.0	林涛	2022-03-30	增加可配置#PERST复位时间说明
V4.4.0	林鼎强	2022-05-24	增加 PM L1 Substates 支持说明

版本号	作者	修改日期	修改说明
V4.5.0	林涛	2022-07-24	增加debugfs的操作说明
V4.6.0	林涛	2022-07-26	增加平台关于cache一致性的说明
V4.7.0	林涛	2022-08-08	增加设备quirks修复说明
V4.8.0	林涛	2022-08-17	增加RK1808说明， 追加debugfs信息
V4.9.0	林涛	2022-08-23	明晰DMA通道情况， 增加3588的5.4内核支持说明
V5.0.0	林涛	2022-08-25	调整MEM与IO BAR异常说明， 增加M.2接口说明， 增加link失败情况
V5.1.0	林涛	2022-08-28	增加Wi-Fi DTS配置范例和休眠唤醒设备状态切换异常
V5.2.0	林涛	2022-11-18	增加RK3528芯片说明以及模块掉电不充分的排查
V5.3.0	林涛	2022-12-14	调整标准EP的说明, 新增PHY时钟模式和各芯片PCIe控制器访问优先级
V5.4.0	林涛	2023-01-03	增调内核版本， 调整部分RK356X说明， 新增资源过滤方法， 明确Lane极性和线序交换
V5.5.0	林涛	2023-02-06	修复RK3528 combo phy时钟调试选项
V5.6.0	薛小明	2023-02-24	修正补充BAR地址访问异常的修复措施
V5.7.0	杨凯	2023-03-14	增加进入物理信号compliance测试模式说明
V5.8.0	杨凯	2023-04-18	删除芯片互联及标准EP的指南， EP模式由 <Rockchip_Developer_Guide_PCIE_EP_Standard_Card> 进行说明
V5.9.0	林涛	2023-05-29	增加NVMe设备温控说明

版本号	作者	修改日期	修改说明
V6.0.0	林涛	2023-06-10	增加Link正常却没有正常枚举出设备的说明
V6.1.0	林涛	2023-06-25	增加PCIe设备唤醒的说明
V6.2.0	林鼎强	2023-07-05	增加 DMATEST、增加几个常见应用说明
V6.3.0	肖垚	2023-07-31	修正PCIe WiFi的REG ON脚的配置说明
V6.4.0	林涛	2023-08-09	统一规范compliance测试的配置
V6.5.0	林鼎强	2023-08-15	修改使用 DMATEST 新版本命令
V6.6.0	林涛	2023-12-19	修正msi-map说明
V6.7.0	杨凯	2024-01-12	增加低速IO信号说明
V6.8.0	林涛	2024-02-26	更新内核版本和新芯片支持
V6.9.0	林鼎强	2024-05-25	更新域地址分配实例
V6.10.0	林鼎强	2024-07-25	优化Lane reversal/Lane polarity说明
V6.11.0	林涛	2024-08-01	添加基于GPIO模式拔插的说明， 进一步明确Lane reversal使用
V6.12.0	林涛	2024-08-22	添加关于下游设备绑定ID的问题说明
V6.13.0	林涛	2024-09-04	添加PCIe PMU perf支持说明
V6.14.0	林涛	2024-09-07	独立出稳定性统计章节，并添加错误注入说明
V6.15.0	林涛	2024-10-18	补充板级可配置的时序
V6.16.0	林涛	2024-11-05	补充NVMe异常处理

版本号	作者	修改日期	修改说明
V6.17.0	林涛	2024-11-19	添加设备休眠不断电的配置；其他配置修复
V6.18.0	林涛	2024-12-03	增加PC模型卡死的问题说明
V6.19.0	林涛	2025-01-12	增加一些性能说明和BAR访问说明
V6.20.0	林涛，林鼎强	2025-02-07	补充限制PCIe带宽的配置、增加非同源时钟方案说明
V6.20.1	林鼎强	2025-02-14	修改引用PCIE_EP_Standard_Card 文档
V6.21.0	林涛	2025-02-18	补充附录中有关PCIe在总线上传输状态的查询，和AI卡问题
V6.22.0	林涛	2025-02-26	补充附录关于PCIe传输阻塞的具体类型和控制器基地址
V6.23.0	林涛	2025-03-14	移除PC模型卡死说明
V6.24.0	林涛	2025-03-17	增加PCIe休眠状态下链路保持的说明
V6.25.0	林涛、林鼎强	2025-05-01	因迁移性能问题到其他文档，删除冗余；增加iommu使用介绍；增加PHY调参说明
V6.26.0	林涛	2025-05-22	增加RK3588拆分使用的注意事；添加AER注入测试支持
V6.27.0	林涛	2025-07-29	修订IO名称；增加32位BAR空间说明；补充PCIe2初始化失败说明；补充NVMe复位补丁
V6.28.0	林涛	2025-09-05	修正L1 substate的部分说明避免歧义；修订删除MSI说明；删除逐个芯片BAR说明
V6.29.0	林涛、林鼎强	2025-11-20	修正AER配置、增加链路训练失败低速IO排查建议
V6.30.0	林涛	2025-12-20	修正L1SS异常配置检查；增加内核版本支持；GKI配置检查
V6.31.0	林涛	2026-01-26	添加RK3572；移除常见问题“可分配的MSI数量”和“处理cache一致性”中芯片列举等
V7.0.0	林涛	2026-03-05	拆分文档，移除问题和异常排查部分，本文档仅保留资源说明和功能配置说明

目录

Rockchip Linux PCIe 开发指南

1. 芯片资源介绍
    - 1.1 RK1808
    - 1.2 RK3528
    - 1.3 RK3562
    - 1.4 RK3566
    - 1.5 RK3568
        - 1.5.1 控制器
        - 1.5.2 PHY
    - 1.6 RK3572
    - 1.7 RK3576
    - 1.8 RK3588
        - 1.8.1 控制器
        - 1.8.2 PHY
    - 1.9 RK3588S
        - 1.9.1 控制器
        - 1.9.2 PHY
2. DTS 配置
    - 2.1 配置要点
    - 2.2 RK1808 DTS配置
    - 2.3 RK3528 DTS配置
    - 2.4 RK356X DTS配置
        - 2.4.1 RK3562 dts
        - 2.4.2 RK3566 dts
        - 2.4.3 RK3568 dts
        - 2.4.4 RK3572 dts
        - 2.4.5 RK3576 dts
    - 2.5 RK3588 DTS配置
        - 2.5.1 示例1 pcie3.0 4Lane RC + 2个pcie 2.0(comboPHY) (RK3588 evb1)
        - 2.5.2 示例2 pcie3.0phy拆分2个2Lane RC, 3个PCIe 2.0 1Lane(comboPHY)
        - 2.5.3 示例3 pcie3.0phy拆分为4个1Lane, 1个使用PCIe 2.0 1 Lane(comboPHY)
        - 2.5.4 开启IOMMU支持
    - 2.6 DTS property说明
        - 2.6.1 控制器dts常用配置
        - 2.6.2 comboPHY dts配置
        - 2.6.3 pcie30phy dts配置
    - 2.7 根据原理图填写DTS
        - 2.7.1 低速IO说明
        - 2.7.2 dts配置方法
    - 2.8 Wi-Fi模块设备树编写范例
3. menuconfig 配置
4. 标准EP功能件开发
5. RC mode PM L1 Substates 支持
6. 基于GPIO方式的拔插检测机制
    - 6.1 硬件要求
    - 6.2 软件要求
    - 6.3 使用限制
7. 内核 DMATEST
8. 内核 稳定性统计信息
9. 内核 错误注入测试支持

10. 内核 AER支持和注入错误测试
11. 内核 PMU perf支持
    11.1 软件与配置
    11.2 使用说明
12. 附录
    12.1 Debugfs导出信息解析表
    12.2 错误注入配置对照表
    12.3 关于PCIe TX加重预设值对照表
    12.4 开发资源获取地址
    12.5 板级可配置时序
    12.6 RK3568 pcie30phy SRNS补丁

1. 芯片资源介绍

芯片中各个控制器所支持分配的BAR空间地址请查阅各芯片TRM手册中PCIe章节的Internal Address Mapping部分，关键词为`PCIe Outbound Memory`，每一个控制器对应的表格都包含了32bit BAR和64 bit BAR的起始地址(Base Address)及其总大小(Address Length)。须知：外设配置空间、IO空间和MEM空间共享这地址段，且MEM空间不支持预取功能。

1.1 RK1808

资源	模式	支持lane拆分	支持DMA	支持IOMMU	支持硬件Cache一致性	支持ASPM	备注	MSI
PCIe Gen2 x 2 lane	RC /EP	否	2个读Channel + 2个写Channel	否	否	L0s/L1	内部时钟	65535

1.2 RK3528

资源	模式	支持lane拆分	支持DMA	支持IOMMU	支持硬件Cache一致性	支持ASPM	备注	MSI
PCIe Gen2 x 1 lane	RC only	否	否	否	否	ALL	内部时钟	32

1.3 RK3562

资源	模式	支持lane拆分	支持DMA	支持IOMMU	支持硬件Cache一致性	支持ASPM	备注	MSI
PCIe Gen2 x 1 lane	RC only	否	否	否	否	ALL	内部时钟	32

1.4 RK3566

资源	模式	支持lane拆分	支持DMA	支持IOMMU	支持硬件Cache一致性	支持ASPM	备注	MSI
PCIe Gen2 x 1 lane	RC only	否	否	否	否	L0s/L1	内部时钟	65535

1.5 RK3568

1.5.1 控制器

资源	模式	支持lane拆分	支持DMA	支持IOMMU	支持硬件Cache一致性	支持ASPM	备注	MSI
PCIe Gen2 x 1 lane	RC only	否	否	否	否	L0s/L1	内部时钟	65535
PCIe Gen3 x 2 lane	RC/EP	1 lane RC+ 1 lane RC	2个读Channel + 2个写Channel	否	否	ALL	外置晶振，仅支持pcie30phy	65535
PCIe Gen3 x 1 lane	RC	1 lane RC	否	否	否	ALL	外置晶振，仅支持pcie30phy	65535

1.5.2 PHY

资源	dts节点	参考时钟	拆分	是否combo
pcie30phy	phy@fe8c0000	外部	2Lane：默认 1Lane + 1Lane：rockchip,bifurcation	pcie专用
combphy2_psq	phy@fe840000	内部/外部	-	与SATA/RGMII combo

说明：

- pcie30phy 2Lane默认配置为PCIe Gen3 x 2 lane，拆分后“PCIe Gen3 x 2 lane”和“PCIe Gen3 x 1 lane” 控制

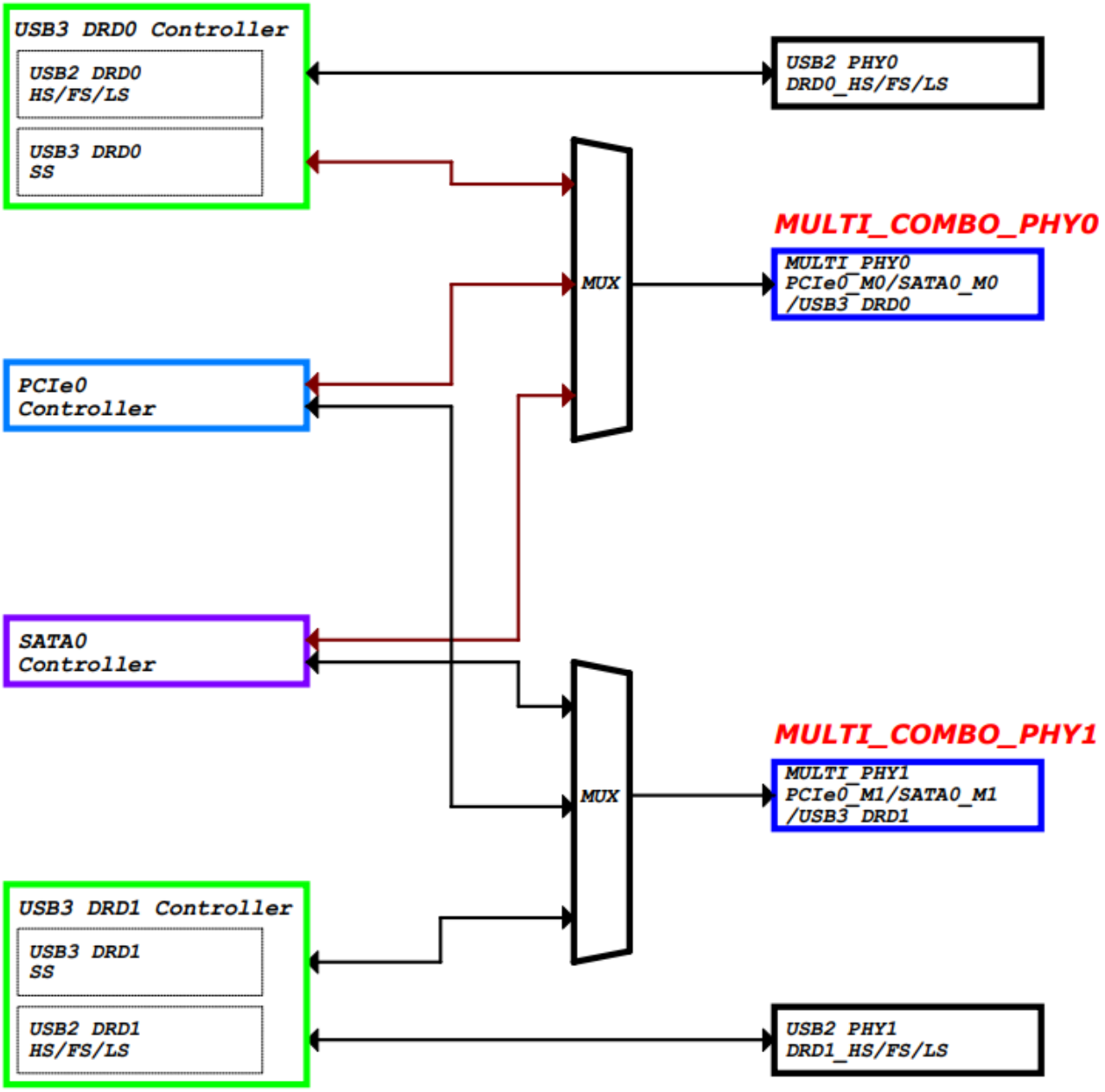
器各1Lane

- pcie30phy支持SRNS No SSC的非同源时钟方案，配置参考附录“RK3568 pcie30phy SRNS补丁“章节，建议通过标准信号测量套件及外设压测确保信号一致性

1.6 RK3572

资源	模式	支持lane拆分	支持DMA	支持IOMMU	支持硬件Cache一致性	支持ASPM	备注	MSI	支持PTM
PCIe Gen2 x 1 lane	RC only	否	否	是	是	ALL	内部时钟	32	是
PCIe Gen2 x 1 lane	RC only	否	否	是	是	ALL	内部时钟	32	是

RK3572的PCIe0控制器支持路由到COMBO_PHY0或者COMBO_PHY1。软件配置路由的方式仅需要修改DTS，让PCIe0的节点引用正确的Phy节点即可。



1.7 RK3576

资源	模式	支持lane拆分	支持DMA	支持IOMMU	支持硬件Cache一致性	支持ASPM	备注	MSI
PCIe Gen2 x 1 lane	RC only	否	否	是	是	ALL	内部时钟	32
PCIe Gen2 x 1 lane	RC only	否	否	是	是	ALL	内部时钟	32

1.8 RK3588

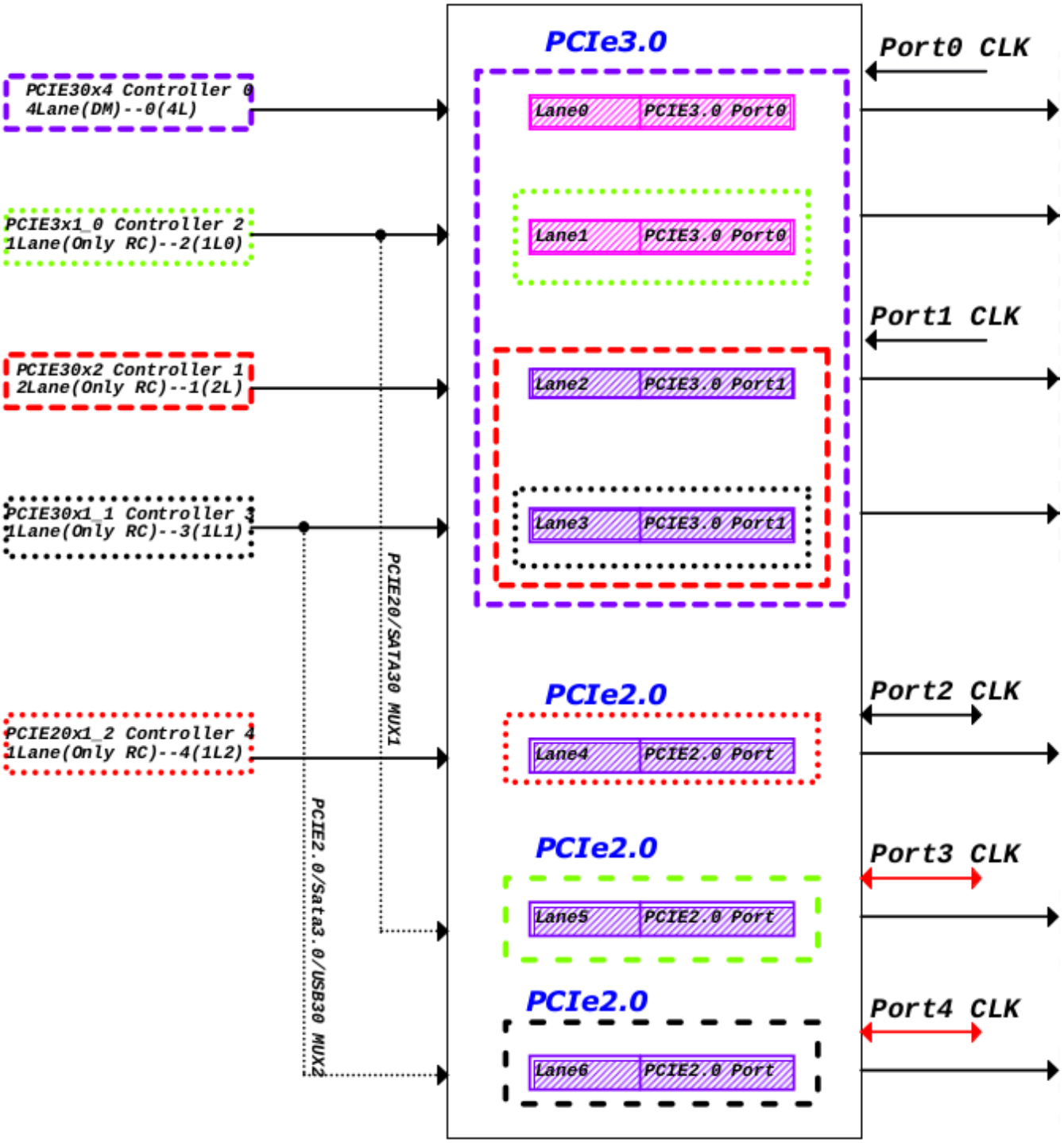
RK3588共有5个PCIe的控制器，硬件IP是一样的，配置不一样，其中一个4Lane DM模式可以支持作为EP使用，另外一个2Lane和3个1Lane控制器均只能作为RC使用。

RK3588有两种PCIe PHY，其中一种为pcie3.0PHY，含2个Port共4个Lane，另一种是pcie2.0的PHY有3个，每个都是2.0 1Lane，跟SATA和USB combo使用。

pcie3.0 PHY的4Lane可以根据实际需求拆分使用，拆分后需要合理配置对应的控制器，所有配置在DTS中完成，无需修改驱动。

使用限制：

1. pcie30phy拆分后，pcie30x4控制器，工作于2Lane模式时只能固定配合pcie30phy的port0，工作于1Lane模式时，只能固定配合pcie30phy的port0lane0；
2. pcie30phy拆分后，pcie30x2控制器，工作于2Lane模式时只能固定配合pcie30phy的port1，工作于1Lane模式时，只能固定配合pcie30phy的port1lane0；
3. pcie30phy拆分为4个1Lane，pcie3phy的port0lane1只能固定配合pcie2x1l0控制器，pcie3phy的port1lane1只能固定配合pcie2x1l1控制器;
4. pcie30x4控制器工作于EP模式，可以使用4Lane模式，或者2Lane模式使用pcie30phy的port0，pcie30phy的port1中2lane可以作为RC配合其他控制器使用。默认使用common clock作为reference clock时，无法实现pcie30phy port0的lane0工作于EP模式，lane1工作于RC模式配合其他控制器使用，因为Port0的两个lane是共用一个输入的reference clock，RC和EP同时使用clock可能会有冲突。
5. RK3588 pcie30phy 如果只使用其中一个port，另一个port也需要供电，refclk等其他信号可接地。


PCIe3.0
PCIE30x4 Controller 0
4Lane(DM)--0(4L)
Lane0
PCIE3.0 Port0
Port0 CLK
PCIE3x1_0 Controller 2
1Lane(Only RC)--2(1L0)
Lane1
PCIE3.0 Port0
Port1 CLK
PCIE30x2 Controller 1
2Lane(Only RC)--1(2L)
Lane2
PCIE3.0 Port1
PCIE30x1_1 Controller 3
1Lane(Only RC)--3(1L1)
Lane3
PCIE3.0 Port1
PCIE20/SATA30 MUX1
PCIe2.0
PCIE20x1_2 Controller 4
1Lane(Only RC)--4(1L2)
Lane4
PCIE2.0 Port
Port2 CLK
PCIe2.0
Lane5
PCIE2.0 Port
Port3 CLK
PCIE2.0/Sata3.0/USB30 MUX2
PCIe2.0
Lane6
PCIE2.0 Port
Port4 CLK

1.8.1 控制器

资源	模式	dts 节点	可用phy	内部DMA	支持ASPM	支持IOMMU	支持硬件Cache一致性	MSI
PCIe Gen3 x 4 lane	RC/EP	pcie3x4: pcie@fe150000	pcie30phy	2个读Channel + 2个写Channel	ALL	是	否	65535
PCIe Gen3 x 2 lane	RC only	pcie3x2: pcie@fe160000	pcie30phy	否	ALL	是	否	65535
PCIe Gen3 x 1 lane	RC only	pcie2x1l0: pcie@fe170000	pcie30phy, combphy1_ps	否	ALL	是	否	65535
PCIe Gen3 x 1 lane	RC only	pcie2x1l1: pcie@fe180000	pcie30phy, combphy2_psu	否	ALL	是	否	65535
PCIe Gen3 x 1 lane	RC only	pcie2x1l2: pcie@fe190000	combphy0_ps	否	ALL	是	否	65535

1.8.2 PHY

资源	dts节点	参考时钟	拆分	是否combo
pcie30phy	phy@fee80000	外部	4Lane*1: PHY_MODE_PCIE_AGGREGATION* *2Lane+2Lane: PHY_MODE_PCIE_NANBNB* *2Lane+1Lane+1Lane:PHY_MODE_PCIE_NANBBI* *1Lane*4: PHY_MODE_PCIE_NABIBI	pcie专用
combphy0_ps	phy@fee00000	内部/外部	-	与SATA combo
combphy1_ps	phy@fee10000	内部/外部	-	与SATA combo
combphy2_psu	phy@fee20000	内部/外部	-	与SATA/USB3 combo

说明：

- pcie30phy不支持非同源时钟方案

1.9 RK3588S

RK3588S的PCIe较为简单，有2个1Lane控制器和2个可用于pcie 2.0的1Lane comboPHY，是一一对应关系。

1.9.1 控制器

资源	模式	dts 节点	可用phy	内部DMA	支持ASPM	支持IOMMU	支持硬件Cache一致性	MSI
PCIe Gen3 x 1 lane	RC only	pcie2x1l1: pcie@fe180000	combphy2_psu	否	ALL	是	否	65535
PCIe Gen3 x 1 lane	RC only	pcie2x1l2: pcie@fe190000	combphy0_ps	否	ALL	是	否	65535

1.9.2 PHY

资源	dts节点	参考时钟	拆分	是否combo
combphy0_ps	phy@fee00000	内部/外部	-	与SATA combo
combphy2_psu	phy@fee20000	内部/外部	-	与SATA/USB3 combo

2. DTS 配置

2.1 配置要点

pcie的配置大部分是固定的，需要在板级dts配置的变量并不多，参考以下要点进行配置即可：

1. 控制器/PHY使能：确定方案后，根据原理图选择使能正确的控制器和PHY，注意控制器的index和phy的index不一定是顺序匹配的，如RK3588的pcie2x1l0不是对应combphy0_ps；
2. 控制器：部分控制器(如RK3588的pcie2x1l0和pcie2x1l1)有不止一个phy可选，按方案设计正确配置“phys”；
3. 控制器：作为RC通常需要配置"reset-gpios", 该项对应原理图PCIE的"PERSTn"信号；
4. 控制器：作为RC可能需要配置"vpcie3v3-supply"，该项对应的是PCIE的"PWREN"gpio信号控制的fixed regulator；
5. 控制器：作为EP使用时，需要修改"compatible"为EP模式对应字串;
6. PHY：pcie30phy共4个lane，可拆分使用，需要根据方案正确配置"rockchip,pcie30-phymode"模式；

2.2 RK1808 DTS配置

RK1808的dts配置，所有的实现模式都在SDK的evb代码中有范例可以参考，可以依照下面表中的模式选择匹配的内容拷贝到产品板级dts中使用。

资源	模式	参考配置	控制器节点	PHY节点	备注
PCIe Gen2 x 2 lane	RC	rk1808-evb.dtsi	pcie0@fc400000	combphy	需要关闭 usbdrd_dwc3 和 usbdrd3
PCIe Gen2 x 2 lane	EP	rk1808-evb.dtsi的pcie0节点添加compatible = "rockchip,rk1808-pcie-ep", "snps,dw-pcie";	pcie0@fc400000	combphy	需要关闭 usbdrd_dwc3和 usbdrd3

2.3 RK3528 DTS配置

RK3528的dts配置，所有的实现模式都在SDK的evb代码中有范例可以参考，可以依照下面表中的模式选择匹配的内容拷贝到产品板级dts中使用。

资源	模式	参考配置	控制器节点	PHY节点	备注
PCIe Gen2 x 1 lane	RC	rk3528-evb2-ddr3-v10.dtsi	pcie2x1@fe4f0000	combphy_pu	-

2.4 RK356X DTS配置

RK356x的dts配置，所有的实现模式都在SDK的evb代码中有范例可以参考，可以依照下面表中的模式选择匹配的内容拷贝到产品板级dts中使用。

2.4.1 RK3562 dts

资源	模式	参考配置	控制器节点	PHY节点
PCIe Gen2 x 1 lane	RC	rk3562-evb1-lp4x-v10.dtsi	pcie2x1@ff500000	combphy_pu

2.4.2 RK3566 dts

资源	模式	参考配置	控制器节点	PHY节点
PCIe Gen2 x 1 lane	RC	rk3566-evb1-ddr4-v10.dtsi	pcie2x1@fe260000	combphy2_psq

2.4.3 RK3568 dts

资源	模式	参考配置	控制器节点	PHY节点
PCIe Gen2 x 1 lane	RC	rk3568-evb2-lp4x-v10.dtsi	pcie2x1@fe260000	combphy2_psq
PCIe Gen3 x 2 lane	RC	rk3568-evb1-ddr4-v10.dtsi	pcie3x2@fe280000	pcie30phy
PCIe Gen3 拆分1 lane + 1 lane	RC	rk3568-evb6-ddr3-v10.dtsi	pcie3x2@fe280000 pcie3x1@fe270000	pcie30phy
PCIe Gen3 x 2 lane	EP	rk3568-iotest-ddr3-v10.dts	pcie3x2@fe280000	pcie30phy

2.4.4 RK3572 dts

资源	模式	参考配置	控制器节点	PHY节点
PCIe Gen2 x 1 lane	RC	rk3572-evb1-v10.dtsi rk3572-evb3-v10.dtsi	pcie0@29d00000	combphy0_ps或 combphy1_psu
PCIe Gen2 x 1 lane	RC	rk3572-evb2-v10.dtsi	pcie1@29d10000	combphy2_ps

2.4.5 RK3576 dts

资源	模式	参考配置	控制器节点	PHY节点
PCIe Gen2 x 1 lane	RC	rk3576-test1.dtsi	pcie0@2a200000	combphy0_ps
PCIe Gen2 x 1 lane	RC	rk3576-test1.dtsi	pcie1@2a210000	combphy1_psu

2.5 RK3588 DTS配置

RK3588的控制器和PHY较多，无法在SDK的evb代码中进行穷举所有组合，按配置要点进行配置即可，这里给出几个典型范例供参考。

注意事项：如果将3.0的PHY进行拆分，3.0 PHY的参考时钟芯片的使能脚需要被所有拆分出来的控制器放在电源节点里面一起控制(包括共同引用vpcie3v3-supply或者将vpcie3v3-supply的节点设置为regulator-always-on和regulator-boot-on)，否则就在硬件上直接默认使能。

2.5.1 示例1 pcie3.0 4Lane RC + 2个pcie 2.0(comboPHY) (RK3588 evb1)

```
/ {
    vcc3v3_pcie30: vcc3v3-pcie30 {
        compatible = "regulator-fixed";
        regulator-name = "vcc3v3_pcie30";
        regulator-min-microvolt = <3300000>;
        regulator-max-microvolt = <3300000>;
        enable-active-high;
        gpios = <&gpio3 RK_PC3 GPIO_ACTIVE_HIGH>;
        startup-delay-us = <5000>;
        vin-supply = <&vcc12v_dcin>;
    };
};

&combphy1_ps {
    status = "okay";
};

&combphy2_psu {
    status = "okay";
};

&pcie2x1l0 {
    phys = <&combphy1_ps PHY_TYPE_PCIE>;
    reset-gpios = <&gpio4 RK_PA5 GPIO_ACTIVE_HIGH>;
    status = "okay";
};

&pcie2x1l1 {
    phys = <&combphy2_psu PHY_TYPE_PCIE>;
    reset-gpios = <&gpio4 RK_PA2 GPIO_ACTIVE_HIGH>;
    pinctrl-names = "default";
    pinctrl-0 = <&rtl8111_isolate>;
    status = "okay";
};

&pcie30phy {
    rockchip,pcie30-phymode = <PHY_MODE_PCIE_AGGREGATION>;
    status = "okay";
};

&pcie3x4 {
    reset-gpios = <&gpio4 RK_PB6 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};
```

2.5.2 示例2 pcie3.0phy拆分2个2Lane RC, 3个PCIe 2.0 1Lane(comboPHY)

```
/ {
    vcc3v3_pcie30: vcc3v3-pcie30 {
        compatible = "regulator-fixed";
        regulator-name = "vcc3v3_pcie30";
        regulator-min-microvolt = <3300000>;
        regulator-max-microvolt = <3300000>;
        enable-active-high;
        gpios = <&gpio3 RK_PC3 GPIO_ACTIVE_HIGH>;
        startup-delay-us = <5000>;
        vin-supply = <&vcc12v_dcin>;
    };
};

&combphy0_ps {
    status = "okay";
};

&combphy1_ps {
    status = "okay";
};

&combphy2_psu {
    status = "okay";
};

&pcie2x1l0 {
    phys = <&combphy1_ps PHY_TYPE_PCIE>;
    reset-gpios = <&gpio4 RK_PA5 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};

&pcie2x1l1 {
    phys = <&combphy2_psu PHY_TYPE_PCIE>;
    reset-gpios = <&gpio4 RK_PA2 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};

&pcie2x1l2 {
    reset-gpios = <&gpio4 RK_PC1 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};

&pcie30phy {
    rockchip,pcie30-phymode = <PHY_MODE_PCIE_NANBNB>;
    status = "okay";
};
```

```
&pcie3x2 {
    reset-gpios = <&gpio4 RK_PB0 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};

&pcie3x4 {
    num-lanes = <2>;
    reset-gpios = <&gpio4 RK_PB6 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};
```

2.5.3 示例3 pcie3.0phy拆分为4个1Lane, 1个使用PCIe 2.0 1 Lane(comboPHY)

须知：

- pcie2x1l0/pcie2x1l1硬件上接入pcie3.0phy对应TX/RX信号，dts关闭combphy1_ps/combphy2_psu节点

参考代码：

```
/ {
    vcc3v3_pcie30: vcc3v3-pcie30 {
        compatible = "regulator-fixed";
        regulator-name = "vcc3v3_pcie30";
        regulator-min-microvolt = <3300000>;
        regulator-max-microvolt = <3300000>;
        enable-active-high;
        gpios = <&gpio3 RK_PC3 GPIO_ACTIVE_HIGH>;
        startup-delay-us = <5000>;
        vin-supply = <&vcc12v_dcin>;
    };
};

&combphy0_ps {
    status = "okay";
};

&pcie2x1l0 {
    phys = <&pcie30phy>;
    reset-gpios = <&gpio4 RK_PA5 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};

&pcie2x1l1 {
    phys = <&pcie30phy>;
    reset-gpios = <&gpio4 RK_PA2 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};
```

```
&pcie2x1l2 {
    reset-gpios = <&gpio4 RK_PC1 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};

&pcie30phy {
    rockchip,pcie30-phymode = <PHY_MODE_PCIE_NABIBI>;
    status = "okay";
};

&pcie3x2 {
    num-lanes = <1>;
    reset-gpios = <&gpio4 RK_PB0 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};

&pcie3x4 {
    num-lanes = <1>;
    reset-gpios = <&gpio4 RK_PB6 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};
```

2.5.4 开启IOMMU支持

- 添加PCIe IOMMU配置

```
# 开启PCIe IOMMU
--- a/arch/arm64/boot/dts/rockchip/rk3588s.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588s.dtsi
@@ -3370,7 +3370,7 @@
mmu600_pcie: iommu@fc900000 {
    compatible = "arm,smmu-v3";
    reg = <0x0 0xfc900000 0x0 0x200000>;
    interrupts = <GIC_SPI 369 IRQ_TYPE_LEVEL_HIGH>,
                 <GIC_SPI 371 IRQ_TYPE_LEVEL_HIGH>,
                 <GIC_SPI 374 IRQ_TYPE_LEVEL_HIGH>,
                 <GIC_SPI 367 IRQ_TYPE_LEVEL_HIGH>;
                 interrupt-names = "eventq", "gerror", "priq", "cmdq-sync";
                 #iommu-cells = <1>;
-                status = "disabled";
+                status = "okay";
};

# 需要开启IOMMU的控制器按需引用iommu-map, 请注意数值匹配msi-map
--- a/arch/arm64/boot/dts/rockchip/rk3588.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588.dtsi
@@ -548,6 +548,7 @@
                 num-viewport = <8>;
```

```
                max-link-speed = <3>;
                msi-map = <0x0000 &its1 0x0000 0x1000>;
+               iommu-map = <0x0000 &mmu600_pcie 0x0000 0x1000>;
                num-lanes = <4>;
                phys = <&pcie30phy>;
                phy-names = "pcie-phy";
@@ -603,6 +604,7 @@
                num-viewport = <8>;
                max-link-speed = <3>;
                msi-map = <0x1000 &its1 0x1000 0x1000>;
+               iommu-map = <0x1000 &mmu600_pcie 0x1000 0x1000>;
                num-lanes = <2>;
                phys = <&pcie30phy>;
                phy-names = "pcie-phy";
@@ -658,6 +660,7 @@
                num-viewport = <4>;
                max-link-speed = <2>;
                msi-map = <0x2000 &its0 0x2000 0x1000>;
+               iommu-map = <0x2000 &mmu600_pcie 0x2000 0x1000>;
                num-lanes = <1>;
                phys = <&combphy1_ps PHY_TYPE_PCIE>;
                phy-names = "pcie-phy";
diff --git a/arch/arm64/boot/dts/rockchip/rk3588s.dtsi
b/arch/arm64/boot/dts/rockchip/rk3588s.dtsi
index 9e19b61..b77a073 100644
--- a/arch/arm64/boot/dts/rockchip/rk3588s.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588s.dtsi
@@ -6319,6 +6319,7 @@
                num-viewport = <4>;
                max-link-speed = <2>;
                msi-map = <0x3000 &its0 0x3000 0x1000>;
+               iommu-map = <0x3000 &mmu600_pcie 0x3000 0x1000>;
                num-lanes = <1>;
                phys = <&combphy2_psu PHY_TYPE_PCIE>;
                phy-names = "pcie-phy";
@@ -6373,6 +6374,7 @@
                num-viewport = <4>;
                max-link-speed = <2>;
                msi-map = <0x4000 &its0 0x4000 0x1000>;
+               iommu-map = <0x4000 &mmu600_pcie 0x4000 0x1000>;
                num-lanes = <1>;
                phys = <&combphy0_ps PHY_TYPE_PCIE>;
                phy-names = "pcie-phy";
```

- 本文档附录所述"开发资源获取地址"中获取Kernel5.10_PCIe_MMU支持.zip 或者Kernel6.1_PCIe_MMU支持.zip，并打上压缩包内的补丁。需知：因Linux内核早期无法支持同时使用多个版本的IOMMU体系，所以本补丁对此做了定制修改。
- 如果无需使用显示图像、多媒体相关IP，也可以不加补丁，关闭config项中的`CONFIG_ROCKCHIP_IOMMU`

2.6 DTS property说明

2.6.1 控制器dts常用配置

1. compatible = <?>;

可选配置项：此项目设置PCIe接口使用的是RC模式还是EP模式。

针对RK3568作为RC功能时，需要配置成compatible = "rockchip,rk3568-pcie", "snps,dw-pcie"; 而如果需要修改成EP模式，则需要修改为compatible = "rockchip,rk3568-pcie-ep", "snps,dw-pcie"; 同理针对RK1808、RK3588，仅需将rk3568字段分别替换为rk1808和rk3588。

2. reset-gpios = <&gpio3 13 GPIO_ACTIVE_HIGH>;`

必须配置项：此项是设置 PCIe 接口的 PERST#复位信号；不论是插槽还是焊贴的设备，请在原理图上找到该引脚，并正确配置。否则很有可能将无法稳定完成链路建立。另需特别提醒，如果将多个lane的PCIe接口拆分，那么所拆分出来的每个节点均需配置不同的PERST#信号线。

3. `num-lanes = <4>;`

可选配置项：此配置设置 PCIe 设备所使用的 lane 数量，已在芯片级的dtsi中配置，默认不需要调整，跟可拆分的phy组合使用时，建议按实际硬件配置。

4. `max-link-speed = <2>;`

可选配置项：此配置设置 PCIe 的带宽版本，1 表示 Gen1，2 表示 Gen2，3表示Gen3。需要注意，此配置与芯片相关，原则上不需要每个板子配置，因此我们在芯片级的dtsi中已配置，仅仅是做为一个测试手段，或者客户板子设计异常后的降级手段。

5. `status = <okay>;`

必须配置项：此配置需要在 pcie控制器节点和对应的 phy 节点同时使能。

6. `vpcie3v3-supply = <&vdd_pcie3v3>;`

可选配置项：用于配置 PCIe 外设的 3V3 供电(原则上我司的硬件参考原理图上将PCIe插槽的12V电源和3V3电源合并控制，所以配置3v3的电源之后，12V电源一并控制)。如果板级针对 PCIe 外设的 3V3 需要控制使能，则如范例所示定义一组对应的 regulator，regulator 的配置请参考 Documentation/devicetree/bindings/regulator/。

另需要特别注意，如果是PCIe3.0的控制器，一般需要外接100M晶振芯片，那么该晶振芯片的供电原则上硬件设计与PCIe外设的3V3共用。所以配置了该项之后，除了确认外设3V3供电之外，还需要确认外置晶振芯片的时钟是否输出正常。一般而言，外置晶振芯片需要一个稳定周期来输出时钟。因此请严格参考时钟芯片的手册中规定的最小数值，并在留有测试余量的基础上，在电源节点中指定startup-delay-us属性的数值。另外针对关闭电源后放电较慢的硬件设计，在电源节点中指定 off-on-delay-us属性的数值，保证上下电操作充分。以rk3568为例，详细范例可参考rk3568-evb1-ddr4-v10.dtsi文件中的vcc3v3_pcie节点。

7. `phys`

可选配置项：用于配置控制器使用的phy的phandle引用，部分控制器可以路由到多个phy(如RK3588的pcie2x1l0和pcie2x1l1)，需要注意的是不同的phy引用方式可能有差异，comboPHY需要同时指定phy的工作模式，具体如下：

```
phys = <&pcie30phy>;
phys = <&combphy1_ps PHY_TYPE_PCIE>;
```

8. `rockchip,bifurcation;`

可选配置项：此为**RK3568芯片**特有配置。可以将pcie3x2的2个lane 拆成两个1个lane的控制器来使用。具体的配置方法就是dts中pcie3x1和pcie3x2控制器节点和pcie30phy都使能，并且pcie3x2和pcie3x1节点中都添加rockchip,bifurcation属性。可参考rk3568-evb6-ddr3-v10.dtsi。否则默认情况下，pcie3x1控制器无法使用。

此时lane0是由pcie3x2控制器使用，lane1是由pcie3x1控制器使用，硬件布板上严格按照我司原理图。另注意，此模式下两个1-lane的控制器必须同时工作在RC模式下。

另外需要特别注意，PCIe 3.0拆分成2个单lane后接两个不同外设，由于晶振及其电源是同一路控制。此时请不要将vpcie3v3-supply配置给其中某一个控制器，否则会造成获取了3v3电压操作权限的这路控制器干扰另一控制器所接的外设的正常初始化。此时应该将vpcie3v3-supply所对应的regulator配置成 regulator-boot-on和regulator-always-on。

9. `prsnt-gpios = <&gpio4 15 GPIO_ACTIVE_LOW>;`

可选配置项：用于驱动识别是否存在外设以及相关外围电路，若检测到有效电平则跳过设备检测流程 。根据PCIe电气化特性协议文档，此gpio为低电平时候表示有设备接入。若板子设计与此相反，可修改为GPIO_ACTIVE_HIGH来标识高电平为有设备接入。该信号用于同套软件支持相同板型带PCIe3的产品和不带PCIe3的产品，避免pcie控制器初始化时发生rcu stall的系统异常；

10. `rockchip,perst-inactive-ms = <500>;`

可选配置项：用于配置设备#PERST复位信号的复位时间，单位为毫秒。根据PCIe Express Card Electromechanical Spec要求，下游设备电源稳定到释放#PERST最小需求为100ms，不配置此项则RK驱动默认配置了200ms。若仍不满足外设工作要求，可以酌情调整，以实测为准。

11. `rockchip,s2r-perst-inactive-ms = <1>;`

可选配置项：用于配置休眠唤醒时设备#PERST复位信号的复位时间，单位为毫秒。若不配置，它的数值等同于`rockchip,perst-inactive-ms`的配置。如果外设在休眠期间不断电，以Wi-Fi为例，则休眠过程中#PERST一直处于复位状态，因此唤醒过程中可以缩短#PERST复位信号的时间，甚至可以配置成0。

12. `rockchip,wait-for-link-ms = <1>;`

可选配置项：用于配置设备#PERST复位信号释放后的等待时间，单位为毫秒。此配置用于部分需要较长时间进行内部初始化的外设，防止因其内部初始化较久而使得系统等待链接超时的情况发生。目前常见的需要此配置的是FPGA和部分AI算力卡。

13. `supports-clkreq;`

可选配置项：仅在 RC mode 下有效，请确认已配置 CLKREQN pinctrl iomux 为 function io 后，如果存在此属性，则指定存在从root port到下游设备的CLKREQN信号路由，并且主机网桥驱动程序可以根据CLKREQN信号的存在进行编程，例如，如果没有CLKREQN信号，则将root port设置为不支持PM L1 Substates。

14. `rockchip,lpbk-master`

特殊调试配置：此配置是针对loopback信号测试，使用PCIe控制器构造模拟loopback master环境，让待测试对端设备进入slave模型，非模拟验证实验室的RX环路需求请勿配置。另注意，Gen3控制器可能需要配置compliance模式，才可以loopback slave模式。如果阅读者不理解什么是loopback测试，说明这不是你要找的配置，请勿针对此配置提问。

15. `rockchip,compliance-mode`

特殊调试配置：此配置是针对compliance信号测试，使PCIe控制器强制进入compliance测试模式或者当使用SMA夹具进入测试模式后不断电。这是一个包含两个配置的数组，数组的第一个配置表示测试模式，第二个配置表示在前述配置下的preset数值。如果使用SMA夹具测试，建议配置`rockchip,compliance-mode=<0 0>;`；如果测试焊接设备，需要固定配置模式和preset数值，`rockchip,compliance-mode=<mode preset>;` 。mode根据需要配置成1、2或者3，分别代表2.5GT、5.0GT和8GT信号。仅在5GT和8GT模式下，preset的有效数值为0到10，分别代表P0到P10不同的协议预加重等配置，可参考附录中“关于PCIe TX加重预设值对照表”的部分。

16. `rockchip,keep-power-in-suspend`

可选配置项：仅在 RC mode 下有效，用于实现在休眠状态下不关闭外设的电源和对其进行复位。允许外设在系统休眠后脱机工作。此模式生效还需要pcie节点引用`vpcie3v3-supply`。

2.6.2 comboPHY dts配置

注意：

- 以下配置不适用于RK1808的combphy节点。
- combphy节点数字表示Mux关系，后缀表示复用关系，p、s、u、q分别表示PCIe、SATA、USB、QSGMII。

1. `rockchip,ext-refclk`

特殊调试配置：首先请注意此配置仅仅针对combophy。默认combphy使用SoC内部时钟方案，以RK356X为例，可参阅rk3568.dtsi节点，默认使用24MHz时钟源。除了24MHz时钟源，还支持25M和100M，仅需要调整assigned-clock-rates = <24000000>数值为所需频率即可。内部时钟源方案成本最优，所以作为SDK默认方案，但combphy仍然预留了外部晶振芯片的时钟源输入选择。如果确实需要使用外部时钟晶振芯片提供时钟的方案，请在板级dts的PCIe控制器用的combphy节点中加入rockchip,ext-refclk，且需要注意在节点中加入assigned-clock-rates = <时钟频率>来指定外部时钟芯片输入的频率，仍然只支持24M,25M,100M三档。

2. `rockchip,enable-ssc`

特殊调试配置：首先请注意此配置仅仅针对PCIe所使用的combphy结点。默认情况下，combophy输出时钟不开启展频。如果用户需要规避一些EMI问题，可尝试在对应的combphy节点加入此配置项，开启SSC。

2.6.3 pcie30phy dts配置

1. `rockchip,pcie30-phymode`

可选配置项：该配置为pcie30phy的组合使用模式，需要合理配置，默认为4Lane共用。详细的可选内容参考`include/dt-bindings/phy/phy-snps-pcie3.h`：

```
/*
 * pcie30_phy_mode[2:0]
 * bit2: aggregation
 * bit1: bifurcation for port 1
 * bit0: bifurcation for port 0
 */
#define PHY_MODE_PCIE_AGGREGATION 4 /* PCIe3x4 */
#define PHY_MODE_PCIE_NANBNB    0   /* P1:PCIe3x2  +  P0:PCIe3x2 */
#define PHY_MODE_PCIE_NANBBI    1   /* P1:PCIe3x2  +  P0:PCIe3x1*2 */
#define PHY_MODE_PCIE_NABINB    2   /* P1:PCIe3x1*2 + P0:PCIe3x2 */
#define PHY_MODE_PCIE_NABIBI    3   /* P1:PCIe3x1*2 + P0:PCIe3x1*2 */
```

2.7 根据原理图填写DTS

2.7.1 低速IO说明

PCIe模块的芯片信号连接，除了数据线和参考时钟差分对，可能还有以下这些低速IO：

低速IO名称	功能	说明
PERSTN	GPIO输出	必选，配置dts "reset-gpios"项
WAKEN	GPIO(PMU域)输入	可选，function驱动注册对应GPIO中断及唤醒源，非PCIe控制器驱动处理
PWREN	GPIO输出	可选，配置dts "vpcie3v3-supply"项
CLKREQN	FUNCTION	可选，支持L1SS时使用，配置dts "supports-clkreq"项
PRSNTN	GPIO输入	可选，配置dts "prsnt-gpios"项

说明：

- CLKREQN使用PCIe的function功能，该信号仅L1SS功能需要使用，否则可以不接，使用时必须添加dts对应属性；
- 功能为GPIO的引脚，请勿在pinctrl里面配置为pcie function，具体用法请参考dts对应配置说明；
- PERSTN信号是协议要求的必选信号，其他信号根据实际项目需求进行配置；
- PCIe EP模式引脚说明参考《Rockchip_Developer_Guide_PCIE_EP_Standard_Card_CN》文档相关章节

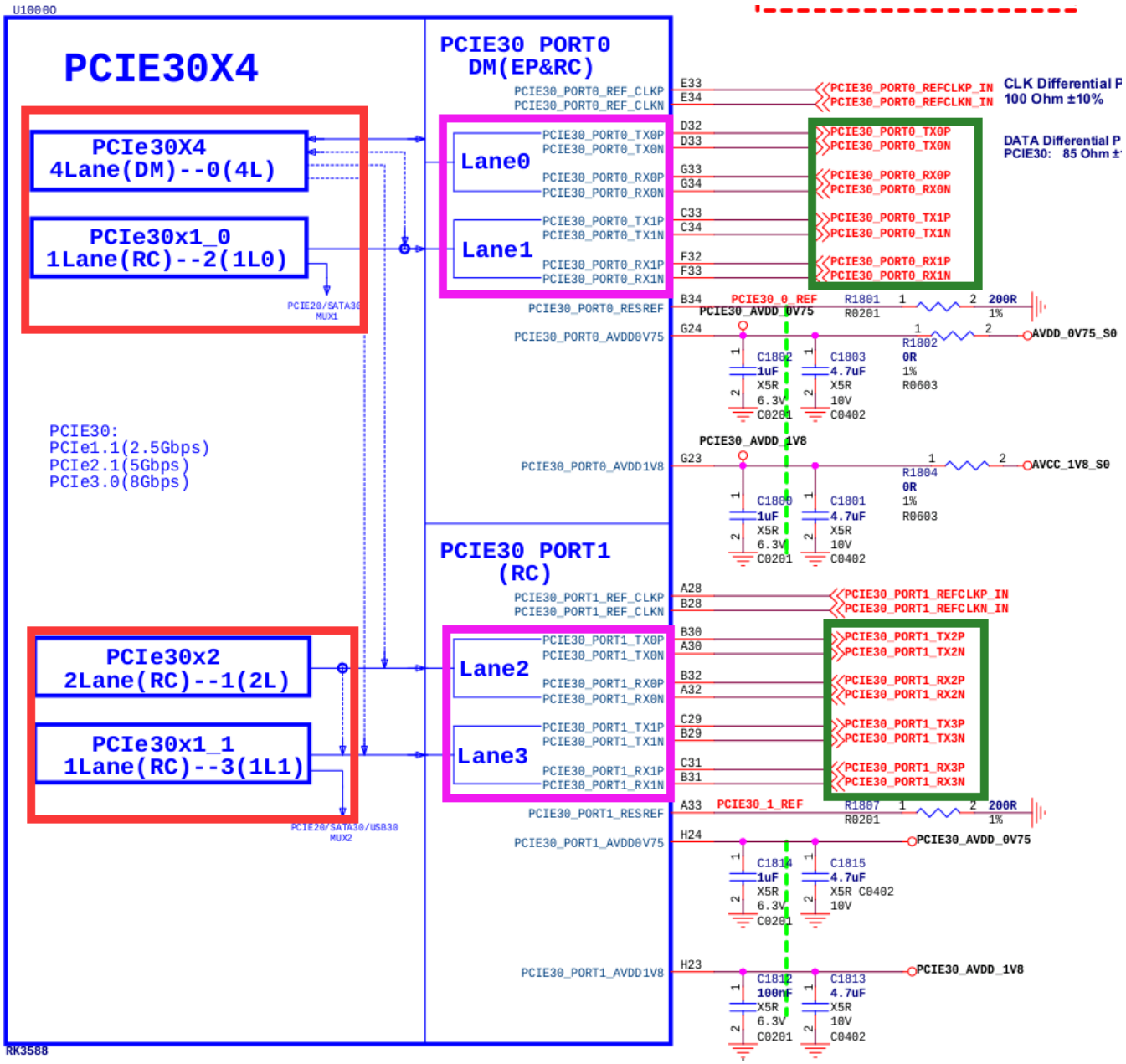
2.7.2 dts配置方法

原理图是基于IO信号的视角来描述硬件，IO信号是跟PHY的index强相关的，前面提到RK3588的controller和PHY的index可能不一致，所以看原理图的时候需要特别注意这一点。这里给出一些填写建议，并通过示例说明如何将原理图中的PHY和控制器对应到dts的节点。

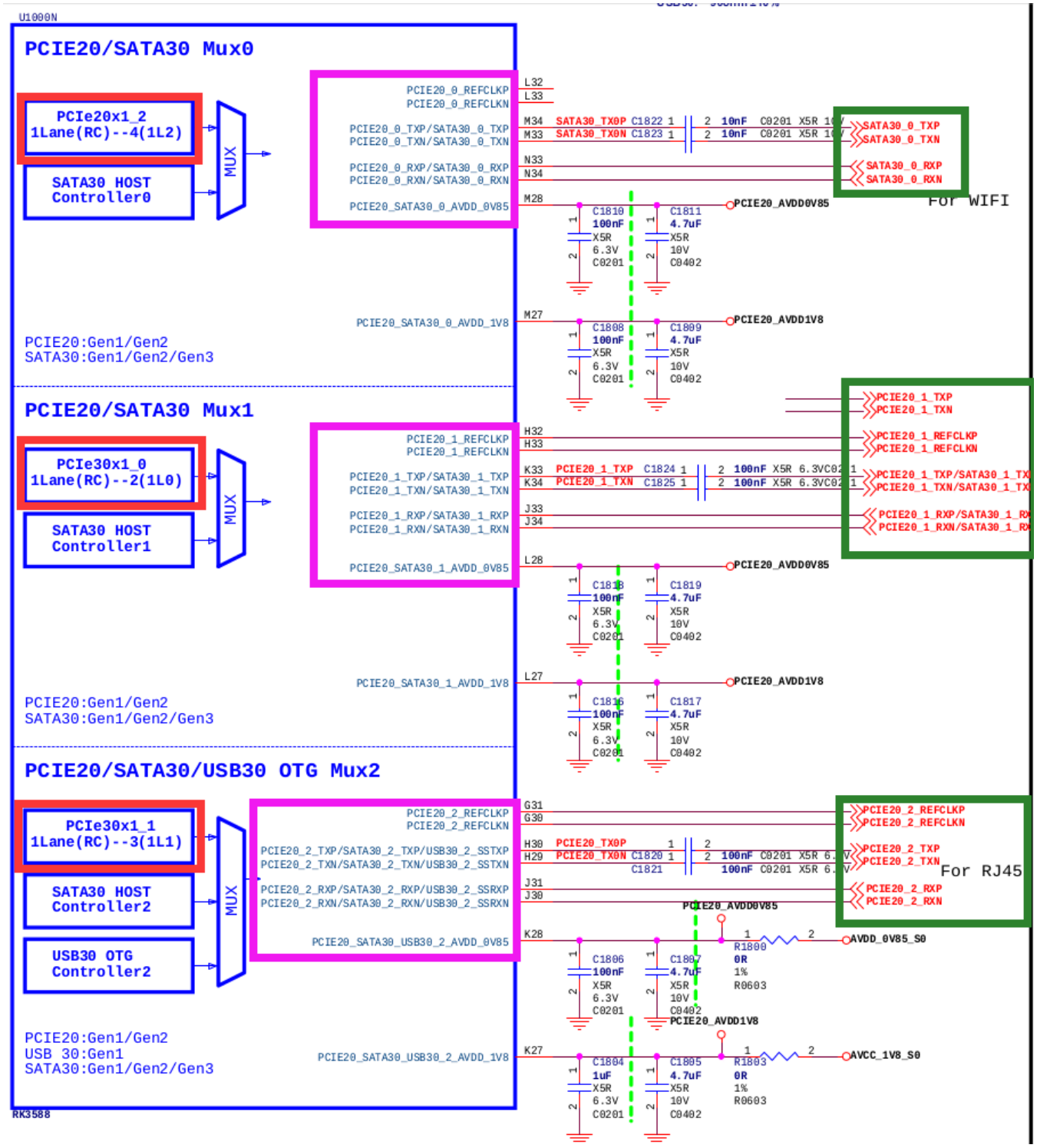
根据硬件原理图来填写dts的建议步骤：

1. 跟硬件工程师确认使用了几个PCIe设备，芯片的多个PCIe接口是如何分配的；
2. 在原理图中分别查找某个设备使用的PCIe数据线对应到哪个PHY的输出；
3. 确定当前设备使用的分别是哪个控制器和PHY，在dts中使能；
4. 确定当前pcie接口使用的控制器dts "phy"属性及模式是否选择正确，如pcie2x1ln控制器需要选择comboPHY且指定为PHY_TYPE_PCIE;
5. comboPHY可能被sata、usb、rgmii等多个控制器共用，请确认对应的其他控制器在dts中被disable；
6. 确定当前phy是否有多种工作模式，配置是否正确，如pcie30phy的不同拆分组合需要正确配置对应模式；
7. 确定当前pcie接口使用的"PERSTn"信号是哪个GPIO，正确配置到控制器dts节点；
8. 确定当前pcie接口使用的"PWREN"信号是哪个GPIO控制的，正确配置到控制器dts节点(这个配置也可以放到on board外设的dts中)；
9. 配置其他外设工作所需的硬件；

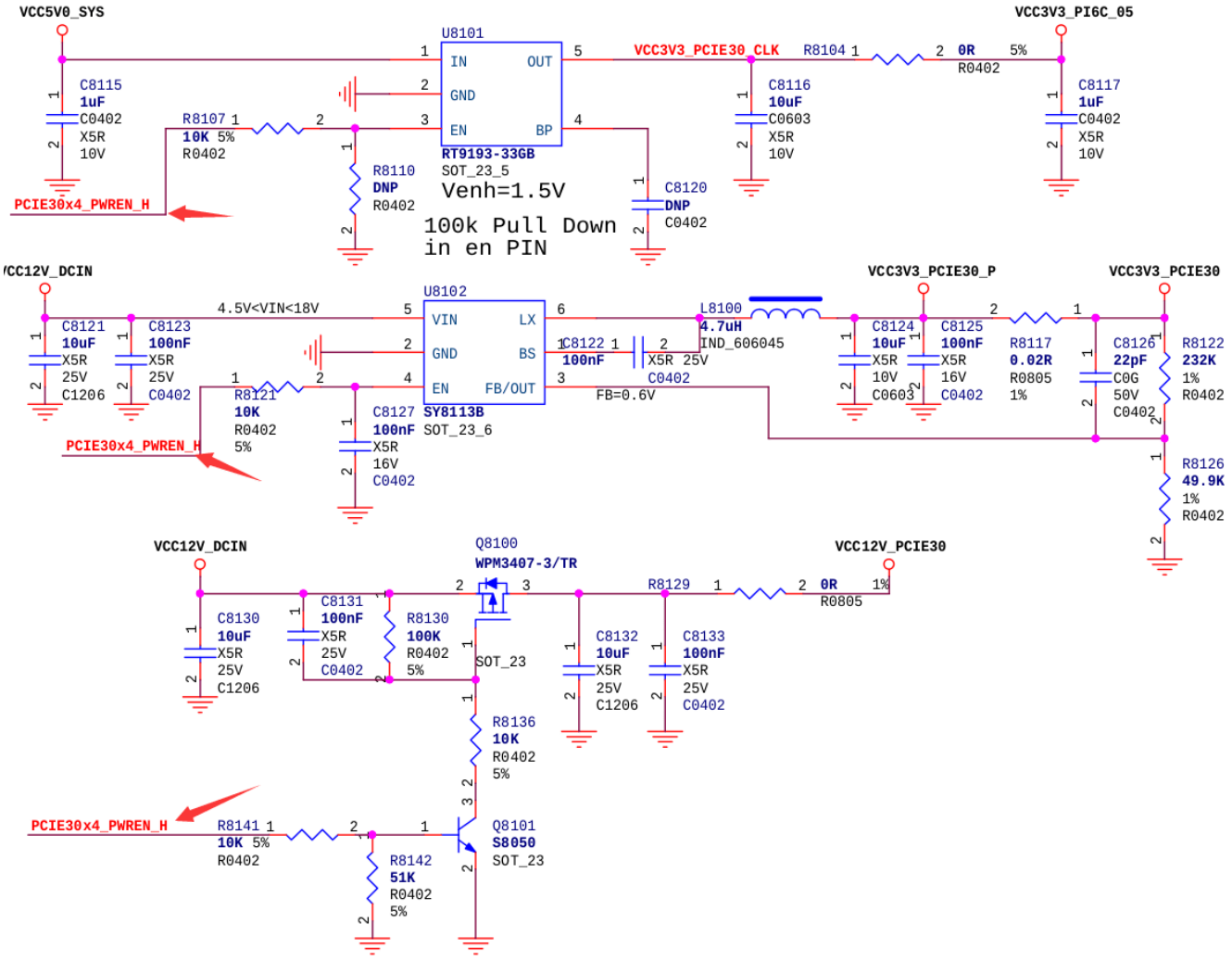
下图是RK3588 pcie30phy及其可能使用的controller，红色方框为controller，粉色方框为PHY信号，绿色方框为外设信号；实际使用哪个控制器可以通过外设信号连接来确认，也可以跟硬件工程师核对理解是否正确。下图是来自RK3588 evb1, 设备接的是一个pcie3.0 x4的slot，所以controller用的是PCIe30X4(dts命名pcie3x4)，其他几个controller都未跟这个PHY配合使用。



下图是RK3588 comboPHY及其可能使用的controller，红色方框为controller，粉色方框为PHY信号，绿色方框为外设信号；实际使用哪个控制器可以通过外设信号连接来确认，也可以跟硬件工程师核对理解是否正确。此图中Mux0的PHY(combphy0_ps)工作于SATA模式，未工作于PCIe；Mux1的PHY(combphy1_ps)配合PCIe30x1_0(dts命名为pcie2x1l0)可能工作于PCIe模式，需要由最终实际接的设备来确定；Mux2的PHY(combphy2_psu)配合PCIe30x1_1(dts命名为pcie2x1l1)工作于pcie模式用于连接一个PCIe网卡。



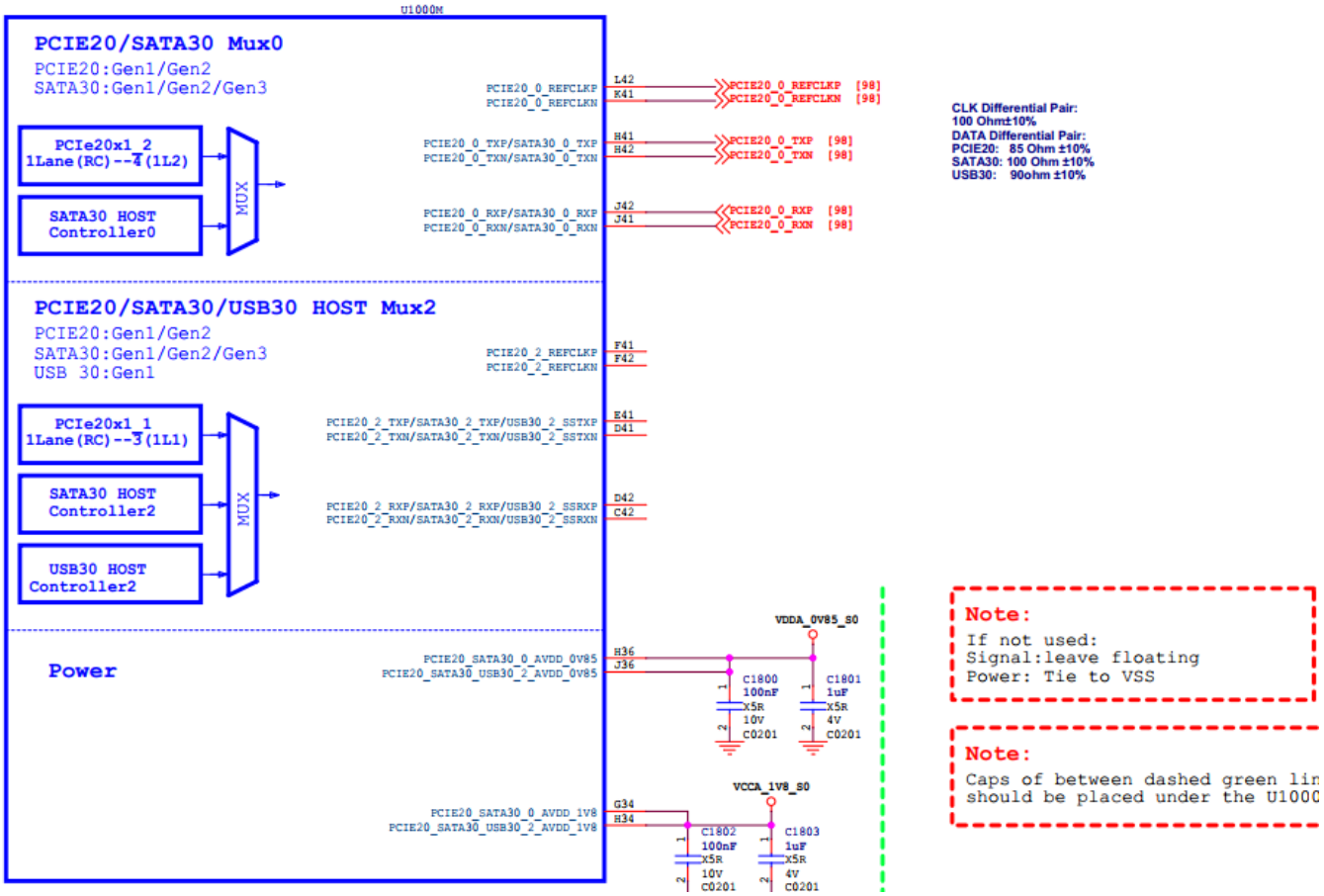
下图为RK3588 evb1的pcie3.0接口供电，可以通过PCIE30X4_PWREN_H这个信号来定位对应的RK3588 GPIO，然后填到pcie3x4控制器dts中。



2.8 Wi-Fi模块设备树编写范例

由于Wi-Fi模块一般接入PCIe2.0口，其使用的combphy复用关系复杂，且其电源使用方式、休眠模式、复位需求与其他设备有明显的不同，此处给出一个设备树编写范例，请酌情参考。

1. 查看原理图，弄清楚Wi-Fi模块所使用的combphy。combphy节点数字表示Mux关系，后缀表示复用关系，p、s、u、q分别表示PCIe、SATA、USB、QSGMII。以此图为例，Wi-FI是接到了PCIe与SATA复用的PCIE20/SATA30 Mux0上，因此配置的应该是combphy0_ps节点。



2. 搜索dts文件，确保复用此combphy功能的其他控制器节点关闭，防止信号干扰。
3. 将wifi_reg_on信号从wireless_wlan节点挪到PCIe 3.3v电源控制节点中。
4. 如果Wi-Fi需要实现L1.x功耗模式，请参考"RC mode PM L1 Substates 支持"章节。
5. 如果Wi-Fi需要实现无线唤醒功能，需要确保 wifi_reg_on管脚在休眠时保持高电平，wifi_host_wake管脚连接到不断电的PMU IO上用于产生中断唤醒主控，具体硬件接发和软件修改详述请参考我司SDK发布文档中的《Rockchip_Developer_Guide_Linux_WIFI_BT_CN.pdf》相关章节。

```
+ vcc3v3_pcie20_wifi: vcc3v3-pcie20-wifi {
+     compatible = "regulator-fixed";
+     regulator-name = "vcc3v3_pcie20_wifi";
+     regulator-min-microvolt = <3300000>;
+     regulator-max-microvolt = <3300000>;
+     enable-active-high;
+    /*
+     * wifi_reg_on 是在vbat、vddio稳定后才使能，在reset-gpios前拉高，所以
+     * 放到PCIe的电源节点中才满足模块时序，引用wifi_poweren_gpio。
+     */
+     pinctrl-0 = <&wifi_poweren_gpio>;
+     startup-delay-us = <5000>;
+     vin-supply = <&vcc12v_dcin>;
+   };

    wireless_wlan: wireless-wlan {
        compatible = "wlan-platdata";
        wifi_chip_type = "ap6275p";
        pinctrl-names = "default";
        pinctrl-0 = <&wifi_host_wake_irq>;
```

```
        WIFI,host_wake_irq = <&gpio0 RK_PA0 GPIO_ACTIVE_HIGH>;
+       /* 注意：这里也需要配置wifi_reg_on管脚，给WiFi驱动来控制 */
+       WIFI,poweren_gpio = <&gpio0 RK_PC7 GPIO_ACTIVE_HIGH>;
        status = "okay";
    };
+
+&sata0 {
+    status = "disabled" /* sata0与pcie2x1l2复用了combphy0_ps, 需确保禁用 */
+}
+
+&combphy0_ps {
+    status = "okay"; /* 确保phy开启 */
+};
+
+&pcie2x1l2 {
+    reset-gpios = <&gpio3 RK_PD1 GPIO_ACTIVE_HIGH>;
+    rockchip,skip-scan-in-resume;
+    rockchip,perst-inactive-ms = <500>; /* 参考Wi-Fi模组手册，查询所需#PERST复位时间 */
+    vpcie3v3-supply = <&vcc3v3_pcie20_wifi>;
+    status = "okay";
+};

&pinctrl {
    wireless-wlan {
        wifi_poweren_gpio: wifi-poweren-gpio {
+           //PCIE REG ON:  务必配置成上拉
+           rockchip,pins = <0 RK_PC7 RK_FUNC_GPIO &pcfg_pull_up>;
        };
    };
};
```

3. menuconfig 配置

1. 需要确保如下配置打开，方可正确的使用 PCIe 相关功能

```
CONFIG_PCI=y
CONFIG_PCI_DOMAINS=y
CONFIG_PCI_DOMAINS_GENERIC=y
CONFIG_PCI_SYSCALL=y
CONFIG_PCI_BUS_ADDR_T_64BIT=y
CONFIG_PCI_MSI=y
CONFIG_PCI_MSI_IRQ_DOMAIN=y
CONFIG_PHY_ROCKCHIP_SNPS_PCIE3=y
CONFIG_PHY_ROCKCHIP_NANENG_COMBO_PHY=y
CONFIG_PCIE_DW=y
CONFIG_PCIE_DW_HOST=y
CONFIG_PCIE_DW_ROCKCHIP=y
CONFIG_PCIEPORTBUS=y
CONFIG_PCIE_PME=y
```

```
CONFIG_GENERIC_MSI_IRQ=y
CONFIG_GENERIC_MSI_IRQ_DOMAIN=y
CONFIG_IRQ_DOMAIN=y
CONFIG_IRQ_DOMAIN_HIERARCHY=y
```

2. 使能 NVMe 设备(建立在 PCIe 接口的 SSD)，PCIe转接AHCI设备（SATA），PCIe转接USB设备(XHCI) 均已在默认config中打开，烦请确认。其他转接设备例如以太网卡，WiFi等请自行确认相关config配置。

```
CONFIG_BLK_DEV_NVME=y
CONFIG_SATA_PMP=y
CONFIG_SATA_AHCI=y
CONFIG_SATA_AHCI_PLATFORM=y
CONFIG_ATA_SFF=y
CONFIG_ATA=y
CONFIG_USB_XHCI_PCI=y
CONFIG_USB_XHCI_HCD=y
```

特别说明，默认内核仅支持 drivers/ata/ahci.c 中列表内的PCIe转接SATA设备，超出部分请找原厂或者代理商支持。

4. 标准EP功能件开发

RK部分芯片的PCIe控制器支持EP模式，可以将芯片开发为标准PCIe EP产品，针对EP功能的实现，请参考文档《Rockchip_Developer_Guide_PCIE_EP_Standard_Card》。

5. RC mode PM L1 Substates 支持

当确认所用 RK 主控及所接的外设都支持 PCIe PM L1 Substates，可以通过开启 PM L1 Substates 功能支持对功耗进一步优化。

进一步强调，如果目标外设中有不支持 PM L1 Substates 的设备，尤其是主板设计为 slot 外接非固定器件，请勿开启 PM L1 Substates 功能，否则部分设备将无法正常工作。

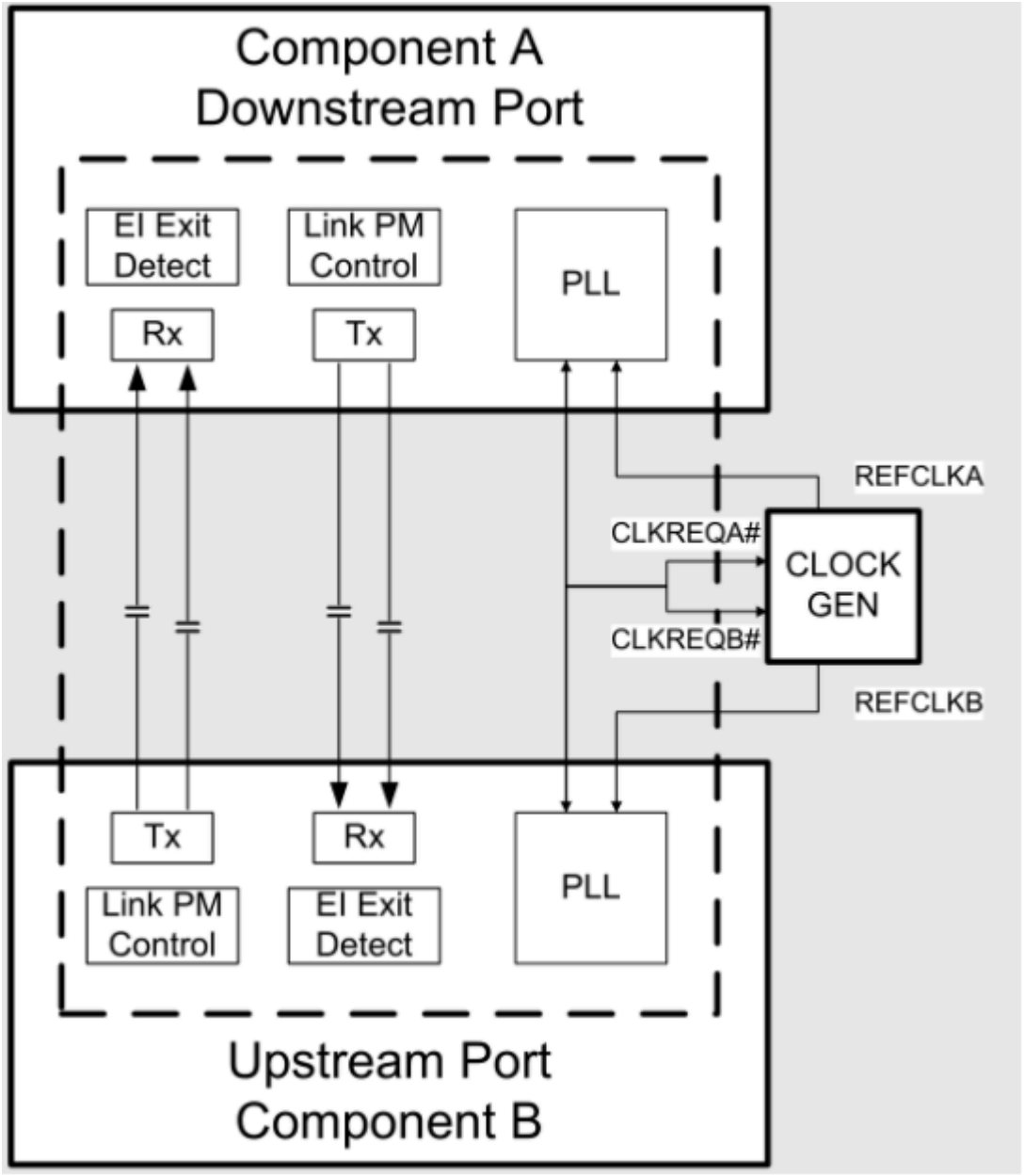
须知

- RK PCIe L1SS特性TL1O_REFCLK_ON时序10us以内，不可调，理论上能兼容大部分外设，但个别外设对该指标约束较为严格，建议和相关外设原厂进一步确认该指标约束情况。
- RK PCIe L1SS退出时间Tpower_on默认为10us，满足自身时序。但是外设的Tpower_on时间不能确保满足其工作时序，如果不满足可能出现链路不稳定。建议和相关外设原厂进一步确认该指标。

PM L1 Substates 支持的硬件电路设计：

- 内部参考时钟方案：RC的CLKREQN与EP的CLKREQN 双端互联
- 外部参考时钟方案：RC的CLKREQN 接入 CLOCK_GEN 的使能脚 OE，如果 CLOCK_GEN 的使能脚 OE 是高电平有效，则应添加反相器
- 内部参考时钟和外部参考时钟方案，CLKREQN都需要设置上拉状态，可以是主控内部设置，也可以用外部上拉

电阻，二者选其一。



系统默认未开启对 PM L1 Substates 支持

- dts 未添加 ”supports-clkreq;“ 属性
- 内核宏配置 CONFIG_PCIEASPM_POWER_SUPERSAVE=n

系统开启 PM L1 Substates 支持

- 确认 RC/EP 都支持 PM L1 Substates，其中RC的支持情况可查阅”芯片资源介绍“章节之ASPM一栏。
- 硬件配置 CLKREQN 信号：确认符合 “PM L1 Substates 支持的硬件电路设计”
- 软件配置 CLKREQN 信号：设置 CLKREQN iomux 为 function io，并确认上拉状态
- 软件配置控制器节点添加 ”supports-clkreq;“ 属性，详细参考 "dts 可配置项 11" 说明
- 开启 PM L1 Substates 支持：
    - 方法1：内核宏配置 CONFIG_PCIEASPM_POWER_SUPERSAVE=y
    - 方法2：部分外设如 WIFI，支持驱动中使能 PM L1 Substates

说明：

- 如果不严格符合以上条件却要尝试通过开启内核宏配置，PCIe link 可能会进入异常状态，无法唤醒
- 请确认内核源码为较新代码，包含支持 L1SS 补丁：commit e18dfa93 PCI: rockchip: dw: Support PM L1 clock removing

6. 基于GPIO方式的拔插检测机制

6.1 硬件要求

1. PCIe slot的PRSNTN_1需要与主控的任意GPIO相连，作为检测脚
2. PCIe设备的电源需要软件可控制上下电

6.2 软件要求

1. 至少包含以下提交，如无请联系业务获取补丁：

```
commit 4de1a0c19e0f9804ba22e7f5e544fea317913957
Author: Shawn Lin <shawn.lin@rock-chips.com>
Date:   Tue Mar 12 16:38:46 2024 +0800

    PCI: rockchip: dw: Add gpio based hotplug
    Change-Id: I49c57755d11cc43bbf7cf9eb23542f5e1e11aaa3

commit 86f3010d7f523c9f5a2e88d9f8f1871ed89da098
Author: Vidya Sagar <vidyas@nvidia.com>
Date:   Sat Oct 1 00:57:45 2022 +0530

    FROMLIST: PCI/hotplug: Add GPIO PCIe hotplug driver
    Change-Id: Iafa798ee4d98f195f5d33d80120da0c569132548
```

2. 内核需确认打开如下配置：

```
CONFIG_HOTPLUG_PCI=y
CONFIG_HOTPLUG_PCI_GPIO=y
```

3. DTS配置参考如下

```
&pcie0 {
  reset-gpios = <&gpio4 RK_PC7 GPIO_ACTIVE_HIGH>;
  vpcie3v3-supply = <&vcc3v3_pcie0>;
  hotplug-gpios = <&gpio4 RK_PC4 IRQ_TYPE_EDGE_BOTH>;
  pinctrl-names = "default";
  pinctrl-0 = <&hot_plug0>;
  status = "okay";
};
```

```
&pinctrl {
  pcie {
      hot_plug0: hot-plug0 {
          rockchip,pins = <4 RK_PC4 RK_FUNC_GPIO &pcfg_pull_up>;
      };
  };
};
```

6.3 使用限制

1. PCIe设备带电拔插极易损坏设备和主控，在设备拔出后的卸载流程和断电流程需要一点时间，所以禁止快速热拔插。需要等到如下移除log后，再重新插入:

```
[   35.680289][  T134] pcieport 0000:00:00.0: Hot-UnPlug Event
[   35.680361][  T134] pcieport 0000:00:00.0: Power Status = 1
[   35.827183][  T134] rk-pcie 2a200000.pcie: rk_pcie_slot_disable
[   35.827303][  T134] pcieport 0000:00:00.0: Hot-UnPlug Event
[   35.827323][  T134] pcieport 0000:00:00.0: Power Status = 0
[   35.827334][  T134] pcieport 0000:00:00.0: Device is already removed
```

2. 为保证数据的完整性和系统的稳定性，需要确保系统停止访问待拔出设备。
3. 无法支持switch下游设备的单独拔插。如有此需求，首先需要确认switch支持下游设备打单独热拔插，然后参考常见应用问题中"如何对下游单个设备进行重扫描或者在线更换设备？"部分进行rescan处理，亦可达到同等效果
4. 不支持在休眠待机状态下，检测插入或者移除设备。

7. 内核 DMATEST

在进行开发前请确认目标PCIe控制器是否支持DMA传输，详细参考“芯片资源介绍”章节。

RK PCIe DMA提供基于内核module_para机制的测试机制，框架类似Linux dmatest，可以基于该框架进一步完成内核下的PCIe DMA应用。

内核版本要求

至少包含以下提交，如无请联系业务获取补丁：

```
commit a7c40cb119703e566d9d5befb8c1a7b0533dd7b7
Author: Jon Lin <jon.lin@rock-chips.com>
Date:   Tue Jan 17 17:46:48 2023 +0800

    PCI: rockchip: dw-dmatest: Suppport rc dma

    1.Set rc dma as default
    2.Changet to ep dma by sending command:
            echo 0 > ./sys/module/pcie_dw_dmatest/parameters/is_rc

    Change-Id: I9b16c328c08f220772e487c7c796b8898d74ae10
    Signed-off-by: Jon Lin <jon.lin@rock-chips.com>
```

测试宏配置

```
CONFIG_PCIE_DW_DMATEST=y
CONFIG_ROCKCHIP_PCIE_DMA_OBJ=n
```

搭建环境

PCIe 互联模型：

- 客户自行搭建RK RC - FPGA EP环境
- RK本地测试搭建为RK RC（RK dmatest 配置）-RK EP（RK 芯片互联配置）

注意：

- MPS 配置为 256B
- RC EP都需要reserved memory，建议在0x3c000000预留64MB测试用内存（测试默认配置的地址）
- 建议关闭其他无关的 PCIe 控制器

测试

详细参考：pcie-dw-misc-dmatest.c源码

```
static int size = 0x20;
module_param(size, int, 0644);
MODULE_PARM_DESC(size, "each packet size in bytes");

static unsigned int cycles_count = 1;
module_param(cycles_count, uint, 0644);
MODULE_PARM_DESC(cycles_count, "how many erase cycles to do (default 1)");

static bool irq;
module_param(irq, bool, 0644);
MODULE_PARM_DESC(irq, "Using irq? (default: false)");

static unsigned int chn_en = 1;
module_param(chn_en, uint, 0644);
MODULE_PARM_DESC(chn_en, "Each bits for one dma channel, up to 2 channels, (default enable
chanel 0)");
```

```
static unsigned int rw_test = 3;
module_param(rw_test, uint, 0644);
MODULE_PARM_DESC(rw_test, "Read/Write test, 1-read 2-write 3-both(default 3)");

static unsigned int bus_addr = 0x3c000000;
module_param(bus_addr, uint, 0644);
MODULE_PARM_DESC(bus_addr, "Dmatest chn0 bus_addr(remote), chn1 add offset 0x100000,
(default 0x3c000000)");

static unsigned int local_addr = 0x3c000000;
module_param(local_addr, uint, 0644);
MODULE_PARM_DESC(local_addr, "Dmatest chn0 local_addr(local), chn1 add offset 0x100000,
(default 0x3c000000)");

static unsigned int test_dev;
module_param(test_dev, uint, 0644);
MODULE_PARM_DESC(test_dev, "Choose dma_obj device,(default 0)");

static bool is_rc = true;
module_param_named(is_rc, is_rc, bool, 0644);
MODULE_PARM_DESC(is_rc, "Test port is rc(default true)");
```

实例参考1：RC设备，channel 0，写数据，数据粒度1MB，循环次数1000，local 地址0x3c000000，remote地址0x3c000000：

```
echo 0 > ./sys/module/pcie_dw_dmatest/parameters/test_dev
echo 1 > ./sys/module/pcie_dw_dmatest/parameters/is_rc
echo 1 > ./sys/module/pcie_dw_dmatest/parameters/chn_en
echo 1 > ./sys/module/pcie_dw_dmatest/parameters/rw_test
echo 0x100000 > ./sys/module/pcie_dw_dmatest/parameters/size
echo 1000 > ./sys/module/pcie_dw_dmatest/parameters/cycles_count
echo 0x3c000000 > ./sys/module/pcie_dw_dmatest/parameters/local_addr
echo 0x3c000000 > ./sys/module/pcie_dw_dmatest/parameters/bus_addr
echo run > ./sys/module/pcie_dw_dmatest/parameters/dmatest
```

实例参考2：EP设备，channel 0/1（双线程同时运行），读写数据，数据粒度8KB，循环次数10000，local 地址0x3c000000，remote 地址 0x3c000000：

```
echo 0 > ./sys/module/pcie_dw_dmatest/parameters/test_dev
echo 0 > ./sys/module/pcie_dw_dmatest/parameters/is_rc
echo 3 > ./sys/module/pcie_dw_dmatest/parameters/chn_en
echo 3 > ./sys/module/pcie_dw_dmatest/parameters/rw_test
echo 0x2000 > ./sys/module/pcie_dw_dmatest/parameters/size
echo 10000 > ./sys/module/pcie_dw_dmatest/parameters/cycles_count
echo 0x3c000000 > ./sys/module/pcie_dw_dmatest/parameters/local_addr
echo 0x3c000000 > ./sys/module/pcie_dw_dmatest/parameters/bus_addr
echo run > ./sys/module/pcie_dw_dmatest/parameters/dmatest
```

8. 内核 稳定性统计信息

PCIe 设备长期工作状态下出现异常，其运行状态不符合预期，可以尝试获取debugfs节点中的信息以便提供给我们进行分析。若需要启用此功能，先确保包含此提交（`pcie: rockchip: dw: Add debugfs support`），否则可以在附录章节所示redmine系统中获取`0001-pcie-rockchip-dw-Add-debugfs-support.patch`。

使用方法：

1. 明确出问题的设备所处的控制器地址节点，可从开机枚举log中查阅或者直接从dtsi中查看。
2. 以`fe16000.pcie`为例, 进入`/sys/kernel/debug/fe160000.pcie`目录
3. `echo disable > err_event` 关闭所有的事件统计功能
4. `echo clear > err_event` 清除所有的事件统计计数
5. `echo enable > err_event` 开启所有的事件统计功能
6. 开始设备老化，复现您的异常case, 复现后请执行`cat dumpfifo`和`cat err_event`。
7. 将导出的信息与本文档附录中的Debugfs导出信息解析表进行对比，可以大致明确出现的问题。

9. 内核 错误注入测试支持

PCIe链路如果需要测试RC端funtion驱动、业务模型/EP端 firmware/双端硬件IP对错误的容错率，可以开启错误注入测试，模拟双方交互过程中可能出现的错误类型，评估双端软件、IP的稳定性。

使用方法：

```
(1) 需要包含以下提交
commit fe835d5fd3329ba629f8c4290c818ef4b8f9895d
Author: Shawn Lin <shawn.lin@rock-chips.com>
Date:   Wed Sep 4 17:04:37 2024 +0800

    PCI: rockchip: dw: Add fault injection support
    Change-Id: Ib214cc1be565bf16bafb6a847215572f35c43753

(2) 需要开启本文档中”内核 稳定性统计信息“章节所述功能，并进入对应控制器的目录

(3) echo "einj_number enable_or_disable error_type error_number" > fault_injection
    |-------------------|-------------------------------------------------------------|
    | 数值               |          含义                                                |
    |-------------------|-------------------------------------------------------------|
    |einj_number:       |错误注入的组号，仅支持0到6，其他数值无效                          |
    |-------------------|-------------------------------------------------------------|
    |enable_or_disable: |开启错误注入还是关闭错误注入，0表示关闭，1表示开启，其他数值无效     |
    |-------------------|-------------------------------------------------------------|
    |error_type:        |错误注入类型选择，根据附录所述选择einj_number所示组号所对应的错误类型|
    |-------------------|-------------------------------------------------------------|
    |error_number:      |错误注入数量，仅支持0到255，其他数值无效                         |
    |-------------------|-------------------------------------------------------------|
    例如echo "2 1 2 128" > fault_injection ,代表开启einj2，注入128个NAK DLLP包
```

```
(4) 启动PCIe链路传输，例如NVMe: dd if=/dev/nvme0n1 of=/dev/null bs=1M count=5000

(5) 查看错误是否发生: cat err_event
    Rx Recovery Request: 0x1f
    ...
    Tx Nak DLLP: 0x80

(6) 分析双端软硬件，查看是否发生预期外的软硬件异常
```

10. 内核 AER支持和注入错误测试

AER(Advanced Error Reporting)功能是Linux框架提供的标准件，用于上报底层通信错误给各类用户态守护进程，建立容错处理机制。在复杂业态环境下，AER注入测试可以提前模拟各类AER事件，完善各类异常处理环节。

1. 打开内核AER和注入支持，将在系统中创建/dev/aer_inject节点

```
CONFIG_PCIEAER=y
CONFIG_PCIEAER_INJECT=y
```

2. 配置内核cmdline

```
#以RK3588-EVB Linux平台为例，添加pcie_pme=nomsi
--- a/arch/arm64/boot/dts/rockchip/rk3588-linux.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588-linux.dtsi
@@ -12,7 +12,7 @@ aliases {
        };

        chosen: chosen {
-               bootargs = "earlycon=uart8250,mmio32,0xfeb50000 console=ttyFIQ0
irqchip.gicv3_pseudo_nmi=0 root=PARTUUID=614e0000-0000 rw rootwait rcupdate.rcu_expedited=1
rcu_nocbs=all";
+               bootargs = "earlycon=uart8250,mmio32,0xfeb50000 console=ttyFIQ0
irqchip.gicv3_pseudo_nmi=0 root=PARTUUID=614e0000-0000 rw rootwait rcupdate.rcu_expedited=1
rcu_nocbs=all pcie_pme=nomsi";
        };
```

3. 确认配置成功并申请到了aer中断服务

```
#系统中查看cmdline是否配置了pcie_pme=nomsi
cat /proc/cmdline
#系统中查看aer中断申请
root@debian:/userdata# cat /proc/interrupts | grep aerdrv
 60:      0      0      0      0      0      0     0     0     INTx   0 Edge      PCIe PME,
aerdrv, PCIe bwctrl
 63:      0      0      0      1      0      0     0     0     INTx   0 Edge      PCIe PME,
aerdrv
110:      0      0      0      0      0      0     0     0     INTx   0 Edge      PCIe PME,
aerdrv
```

4. 本文档附录所述"开发资源获取地址"中获取AER注入工具：aer-inject

5. 查询所需测试的PCIe设备地址

```
使用lspci获取所需测试的PCIe设备地址，以01: 00.0 这个网卡为例
00:00.0 PCI bridge: Rockchip Electronics Co., Ltd Device 3576 (rev 01)
01:00.0 Network controller: Broadcom Inc. and subsidiaries Device 449d (rev 02)
```

6. 编写测试脚本(范例)

```
#脚本名以aer.txt为例，内容如下:

AER
BUS 1 DEV 0 FN 0         //BUS DEV FN分别对应待测设备的BDF
COR_STATUS BAD_TLP       //测试可恢复错误，类型bad TLP
HEADER_LOG 0 1 2 3

AER
BUS 1 DEV 0 FN 0         //BUS DEV FN分别对应待测设备的BDF
UNCOR_STATUS COMP_ABORT //测试不可恢复错误，类型completion abort
HEADER_LOG 4 5 6 7
```

可恢复错误的类型除了BAD_TLP，还可以是RCVR、BAD_TLP、BAD_DLLP、REP_ROLL、REP_TIMER；不可恢复错误除了COMP_ABORT，还可以是TRAIN、DLP、POISON_TLP、FCP、COMP_TIME、COMP_ABORT、UNX_COM、RX_OVE、MALF_TLP、ECRC、UNSUP。一个脚本可以执行各种不同的注入配置，请按需编写。

7. 执行测试脚本

```
root@rk3576-buildroot:/data# ./aer-inject aer.txt
[  459.689869] pcieport 0000:00:00.0: aer_inject: Injecting errors 00000040/00000000 into
device 0000:01:00.0
[  459.690302] pcieport 0000:00:00.0: AER: Corrected error message received from
0000:01:00.0
[  459.690376] pci 0000:01:00.0: PCIe Bus Error: severity=Corrected, type=Data Link Layer,
(Receiver ID)
[  459.690404] pci 0000:01:00.0:   device [14e4:449d] error status/mask=00000040/00002000
[  459.690434] pci 0000:01:00.0:    [ 6] BadTLP
```

11. 内核 PMU perf支持

11.1 软件与配置

```
(1)需包含以下五个提交:
commit 0270f32f207f5682a729c17e977eb87bba83823e
Author: Shuai Xue <xueshuai@linux.alibaba.com>
Date:   Fri Dec 8 10:56:50 2023 +0800

    UPSTREAM: PCI: Move pci_clear_and_set_dword() helper to PCI header
    Change-Id: I35125190a4dd8ba25e6ec14b4712750605c22285

commit 1b627c690ade9a72e3cd488e2e11edffb5d0e879
Author: Shuai Xue <xueshuai@linux.alibaba.com>
Date:   Fri Dec 8 10:56:49 2023 +0800

    UPSTREAM: PCI: Add Alibaba Vendor ID to linux/pci_ids.h
    Change-Id: I86188f119a42548ab777df0449f7d0a933f34d12

commit dcfa6c8947baeac74ab44ea8f03d3831a062c14b
Author: Shuai Xue <xueshuai@linux.alibaba.com>
Date:   Fri Dec 8 10:56:51 2023 +0800

    BACKPORT: drivers/perf: add DesignWare PCIe PMU driver
    Change-Id: I470f4dc2791168760517c77dd31a4dacd7dab591

commit 6cb6a00862fa29f815412634569e2015f86e397a
Author: Shawn Lin <shawn.lin@rock-chips.com>
Date:   Tue Sep 3 16:24:36 2024 +0800

    perf/dwc_pcie: Add support for Rockchip vendor devices
    Change-Id: I6fde80440d2fa058b38a7d927eb846f477812b5f

commit 50cb3fcd18fb9defe23ba95eb3962a287e957166
Author: Shawn Lin <shawn.lin@rock-chips.com>
Date:   Tue Sep 3 16:24:36 2024 +0800

    PCI: rockchip: dw: Add dwc pmu support for rockchip
    Change-Id: Ia27ee055aa3e63deeb7fd646411c3542b7019288

(2)内核需要开启CONFIG_PERF_EVENTS配置
(3)系统需要集成perf工具；若无，可以去本文档附录所述开发资源获取地址内下载。
```

11.2 使用说明

```
(1) 列出所有DWC PCIe PMU支持的配置
root@rk3576-buildroot:/# /userdata/perf list | grep dwc_rootport
  dwc_rootport_0/CFG_RCVRY/                          [Kernel PMU event] #链路rcvry时间占比
  dwc_rootport_0/L0/                                 [Kernel PMU event] #链路处于L0占比
  dwc_rootport_0/L1/                                 [Kernel PMU event] #链路处于L1占比
  dwc_rootport_0/L1_1/                               [Kernel PMU event] #链路处于L1.1占比
  dwc_rootport_0/L1_2/                               [Kernel PMU event] #链路处于L1.2占比
  dwc_rootport_0/L1_AUX/                             [Kernel PMU event] #RK不支持状态
  dwc_rootport_0/RX_L0S/                             [Kernel PMU event] #RX处于L0s占比
  dwc_rootport_0/Rx_CCIX_TLP_Data_Payload/           [Kernel PMU event] #RK不支持CCI数据统计
  dwc_rootport_0/Rx_PCIe_TLP_Data_Payload/           [Kernel PMU event] #RX TLP数据量
  dwc_rootport_0/TX_L0S/                             [Kernel PMU event] #TX处于L0s占比
  dwc_rootport_0/TX_RX_L0S/                          [Kernel PMU event] #TX/RX都处于L0s占比
  dwc_rootport_0/Tx_CCIX_TLP_Data_Payload/           [Kernel PMU event] #RK不支持CCI数据统计
  dwc_rootport_0/Tx_PCIe_TLP_Data_Payload/           [Kernel PMU event] #TX TLP数据量
  dwc_rootport_0/one_cycle/                          [Kernel PMU event] #RK不支持
  dwc_rootport_0/rx_ack_dllp,lane=?/                 [Kernel PMU event] #RX回复的DLLP数
  dwc_rootport_0/rx_atomic,lane=?/                   [Kernel PMU event] #RK不支持
  dwc_rootport_0/rx_ccix_tlp,lane=?/                 [Kernel PMU event] #RK不支持
  dwc_rootport_0/rx_completion_with_data,lane=?/     [Kernel PMU event] #RX cplt包带数据
  dwc_rootport_0/rx_completion_without_data,lane=?/  [Kernel PMU event] #RX cplt包不带数据
  dwc_rootport_0/rx_duplicate_tl,lane=?/             [Kernel PMU event] #RX/TL dup错误数
  dwc_rootport_0/rx_io_read,lane=?/                  [Kernel PMU event] #RX上ior包数
  dwc_rootport_0/rx_io_write,lane=?/                 [Kernel PMU event] #RX上iow包数
  dwc_rootport_0/rx_memory_read,lane=?/              [Kernel PMU event] #RX上memr包数
  dwc_rootport_0/rx_memory_write,lane=?/             [Kernel PMU event] #RX上memw包数
  dwc_rootport_0/rx_message_tlp,lane=?/              [Kernel PMU event] #RX上收到的msg数
  dwc_rootport_0/rx_nulified_tlp,lane=?/             [Kernel PMU event] #RX上因错丢弃的TLP数
  dwc_rootport_0/rx_tlp_with_prefix,lane=?/          [Kernel PMU event] #RX上带前缀的TLP数
  dwc_rootport_0/rx_update_fc_dllp,lane=?/           [Kernel PMU event] #RX上收到的流控包数
  dwc_rootport_0/tx_ack_dllp,lane=?/                 [Kernel PMU event] #TX回复的DLLP数
  dwc_rootport_0/tx_atomic,lane=?/                   [Kernel PMU event] #RK不支持
  dwc_rootport_0/tx_ccix_tlp,lane=?/                 [Kernel PMU event] #RK不支持
  dwc_rootport_0/tx_completion_with_data,lane=?/     [Kernel PMU event] #TX cplt包带数据
  dwc_rootport_0/tx_completion_without_data,lane=?/  [Kernel PMU event] #TX cplt包不带数据
  dwc_rootport_0/tx_configuration_read,lane=?/       [Kernel PMU event] #TX cfg-r包数
  dwc_rootport_0/tx_configuration_write,lane=?/      [Kernel PMU event] #TX cfg-w包数
  dwc_rootport_0/tx_io_read,lane=?/                  [Kernel PMU event] #TX上ior包数
  dwc_rootport_0/tx_io_write,lane=?/                 [Kernel PMU event] #TX上iow包数
  dwc_rootport_0/tx_memory_read,lane=?/              [Kernel PMU event] #TX上memr包数
  dwc_rootport_0/tx_memory_write,lane=?/             [Kernel PMU event] #TX上memw包数
  dwc_rootport_0/tx_message_tlp,lane=?/              [Kernel PMU event] #TX上收到的msg数
  dwc_rootport_0/tx_nulified_tlp,lane=?/             [Kernel PMU event] #TX上因错丢弃的TLP数
  dwc_rootport_0/tx_tlp_with_prefix,lane=?/          [Kernel PMU event] #TX发出的带前缀TLP数
  dwc_rootport_0/tx_update_fc_dllp,lane=?/           [Kernel PMU event] #TX三发出的流控包数

(2) 启动某项perf功能，以基于时间的统计RX TLP为例
 /userdata/perf stat -a -e dwc_rootport_0/Rx_PCIe_TLP_Data_Payload/

(3) 启动传输
```

```
root@rk3576-buildroot:/# dd if=/dev/nvme0n1 of=/dev/null bs=1M count=5000
dd if=/dev/nvme0n1 of=/dev/null bs=1M count=5000
5000+0 records in
5000+0 records out
5242880000 bytes (5.2 GB, 4.9 GiB) copied, 14.9016 s, 352 MB/s

(4) 查看统计
Performance counter stats for 'system wide':
5221423060      dwc_rootport_0/Rx_PCIe_TLP_Data_Payload/     (50.01%)
28.298528222 seconds time elapsed

(5) 同理可以测试TX TLP的数据量，则平均的RX/TX带宽计算
PCIe RX Bandwidth = Rx_PCIe_TLP_Data_Payload / 统计时长
PCIe TX Bandwidth = Tx_PCIe_TLP_Data_Payload / 统计时长

(6) Lane事件的统计
因为每个Lane都拥有相同的事件，为了避免产生大量冗余信息，建议指定Lane ID，例如：
 /userdata/perf stat -a -e dwc_rootport_0/rx_memory_read,lane=1/
```

12. 附录

12.1 Debugfs导出信息解析表

事件符号	含义
EBUF Overflow	-
EBUF Under-run	-
Decode Error	包解码错误，信号异常
Running Disparity Error	极性偏差错误，8bit/10bit编码中0与1数量比例失衡，Gen2信号异常
SKP OS Parity Error	SKP 序列奇偶校验错误，Gen3信号异常
SYNC Header Error	异步包头错误，信号问题
CTL SKP OS Parity Error	控制的SKP 序列奇偶校验错误，Gen3信号异常
Detect EI Infer	检测到信号串扰
Receiver Error	接收端错误
Rx Recovery Request	RX信号收到外设请求，进入rescoery状态进行信号矫正
N_FTS Timeout	外设的n_fts不达标，L0s回到L0异常进入recovery状态
Framing Error	帧格式解码错误，信号异常
Deskew Error	deskew错误，信号异常

事件符号	含义
BAD TLP	收到错误TLP包
LCRC Error	link CRC，信号异常
BAD DLLP	错误的DLLP包，信号异常
Replay Number Rollover	重传包积累太多，replay buffer冲了
Replay Timeout	误码包重传超时。
Rx Nak DLLP	收到来自下游的DLLP而拒绝其访问
Tx Nak DLLP	发往下游的DLLP被拒绝
Retry TLP	重传的TLP包数量，可以表征误码率
FC Timeout	更新流控超时
Poisoned TLP	上报一个错误的TLP类型，可能是因为链路原因或RAM的位跳变引起
ECRC Error	end CRC，信号异常
Unsupported Request	收到不支持的请求类型的TLP包，可能是访问外设被拒绝
Completer Abort	设备接收到了RC发来的请求，但是发生了其内部错误，要求终止该访问请求。
Completion Timeout	读取外设相关资源，TLP返回超时
Common event signal status	读取目前控制器的PM模式

12.2 错误注入配置对照表

组号 einj_number	组描述	类型号 error_type	类型含义
0	CRC错误	0x0	TLP LCRC错包
0	CRC错误	0x1	16b CRC of ACK/NAK DLLP 错包
0	CRC错误	0x2	16b CRC of Update-FC DLLP错包
0	CRC错误	0x3	TLP ECRC错包
0	CRC错误	0x4	TLP FCRC错包(128b/130bit编码)
0	CRC错误	0x5	TX极性错误TSOS(128b/130bit编码)
0	CRC错误	0x6	TX极性错误SKPOS(128b/130bit编码)
0	CRC错误	0x8	RX LCRC错包
0	CRC错误	0xb	RX ECRC错包
1	包编号顺序错误	0x0	TLP SEQ#错误
1	包编号顺序错误	0x1	DLLP SEQ#错误 ACK_NAK_DLLP
2	DLLP错误	0x0	阻止传输DLLP ACK/NAK 包[1]
2	DLLP错误	0x1	阻止传输Update FC DLLP包[1]
2	DLLP错误	0x2	总是发送NAK DLLP包[1]
3	符号错误	0x0	Invert sync header(128bit/130bit编码)
3	符号错误	0x1	TS1 order set错误
3	符号错误	0x2	TS2 order set错误
3	符号错误	0x3	FTS order set错误
3	符号错误	0x4	E-idle order set错误
3	符号错误	0x5	END/EDB symbol错误
3	符号错误	0x6	STP/SDP symbol错误
3	符号错误	0x7	SKP order set错误
4	流控FC错误	0x0	Posted TLP header credit
4	流控FC错误	0x1	Non-Posted TLP header credit

组号 einj_number	组描述	类型号 error_type	类型含义
4	流控FC错误	0x2	Commpletion TLP header credit
4	流控FC错误	0x4	Posted TLP Data credit
4	流控FC错误	0x5	Non-Posted TLP Data credit
4	流控FC错误	0x6	Commpletion TLP data credit
5	特殊TLP错误	0x0	将ACK DLLP当成NAK DLLP来产生duplicate TLP包
5	特殊TLP错误	0x1	制造无效TLP包，原始包放入retry流程
6	包头检查错误	0x0	产生TLP header错误
6	包头检查错误	0x1	产生TLP prefix 中第一组四个dword错误
6	包头检查错误	0x2	TLP prefix 中第二组dword错误

[1] 不依赖于计数到零而停止，需要命令手动停止

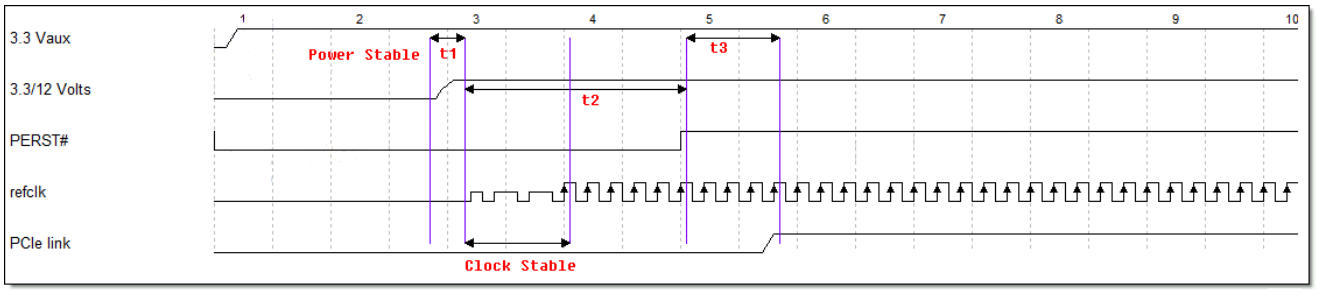
12.3 关于PCIe TX加重预设值对照表

Preset Number	Preshoot(dB)	De-emphasis(dB)
P0	0	-6.0 ± 1.5 dB
P1	0	-3.5 ± 1 dB
P2	0	-4.4 ± 1.5 dB
P3	0	-2.5 ± 1 dB
P4	0	0
P5	1.9 ± 1 dB	0
P6	2.5 ± 1 dB	0
P7	3.5 ± 1 dB	-6.0 ± 1.5 dB
P8	3.5 ± 1 dB	-3.5 ± 1 dB
P9	3.5 ± 1 dB	0
P10	0	x

12.4 开发资源获取地址

取得redmine权限后可以直接访问https://redmine.rock-chips.com/documents/107获取前文所述各类资料。

12.5 板级可配置时序



可调时间	图标	dts属性	说明（Linux 内核阶段）
Power Stable	t1	startup-delay-us	此属性是配置到PCIe节点的vpcie3v3-supply所引用的电源节点中的。
Tpvperl	t2	rockchip,perst-inactive-ms	此属性是配置到PCIe节点中的，若不配置默认为200ms，在开机时候生效。
Tpvperl	t2	rockchip,s2r-perst-inactive-ms	此属性是配置到PCIe节点中的，若不配置默认等同于rockchip,perst-inactive-ms的配置数值，在休眠唤醒时生效。
out of electrical idle	t3	rockchip,wait-for-link-ms	此属性是配置到PCIe节点中的，若不配置默认为1ms, 用于等待每次发起link之前的等待时间。

12.6 RK3568 pcie30phy SRNS补丁

```
diff --git a/drivers/phy/rockchip/phy-rockchip-snps-pcie3.c b/drivers/phy/rockchip/phy-
rockchip-snps-pcie3.c
index 2392bcb..fa3244e 100644
--- a/drivers/phy/rockchip/phy-rockchip-snps-pcie3.c
+++ b/drivers/phy/rockchip/phy-rockchip-snps-pcie3.c
@@ -19,12 +19,15 @@
 #include <dt-bindings/phy/phy.h>

 #define GRF_PCIE30PHY_CON4 0x10
+#define GRF_PCIE30PHY_CON5 0x14
 #define GRF_PCIE30PHY_CON6 0x18

@@ -97,6 +100,17 @@
                (0x1 << 15) | (0x1 << 31));
```

```
    }

+
+   /* rx0_cmn_refclk_mode disabled */
+   regmap_write(priv->phy_grf, GRF_PCIE30PHY_CON5, (0x0) | (0x1 << 25));
+   /* rx1_cmn_refclk_mode disabled */
+   regmap_write(priv->phy_grf, GRF_PCIE30PHY_CON6, (0x0) | (0x1 << 25));
+
    regmap_write(priv->phy_grf, GRF_PCIE30PHY_CON4,
             (0x0 << 14) | (0x1 << (14 + 16))); //sdram_ld_done
    regmap_write(priv->phy_grf, GRF_PCIE30PHY_CON4,
```